



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

SPEECH/MUSIC CLASSIFICATION

DETEKCE ŘEČI A HUDBY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

MAREK HRIC

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. VLADIMÍR MALENOVSKÝ, Ph.D.

BRNO 2026

Bachelor's Thesis Assignment



Institut: Department of Computer Graphics and Multimedia (DCGM)
Student: **Hric Marek**
Programme: Information Technology
Title: **Speech/Music Classification**
Category: Speech and Natural Language Processing
Academic year: 2025/26

172681

Assignment:

1. Study the principles of speech/music classification. Focus primarily on methods for automatic real-time classification of conversational signals, with attention to the distinctive characteristics of such signals.
2. Review and analyze the available literature and resources related to the classification of speech, music, and noise in audio recordings, including an overview of commonly used methods and techniques.
3. Familiarize yourself with current approaches to distinguishing speech, music, and noise – for example, methods such as GMM/HMM, SVM, neural networks, decision trees, and others. Evaluate their strengths and weaknesses based on published results.
4. Choose an appropriate existing method or propose your own approach for the classification of speech, music, and noise. If necessary, combine the described techniques or suggest improvements to them.
5. Implement the selected classifier and conduct experiments with its variants. Consider and describe the modifications you make to the methods with the aim of increasing the solution's effectiveness.
6. Evaluate the results achieved by your classifier in comparison with existing methods on a sufficiently diverse test data set. Prepare a discussion of the results, including possible directions for further development and the potential for follow-up work.
7. Prepare a concise poster and a short video presentation that introduce your bachelor's thesis, its objectives, and main results.

Literature:

1. Banriskhem K., Khonglah. "Speech / music classification using speech-specific features." *Digital Signal Processing*, Volume 48, January 2016, pp. 71–83. DOI: [10.1016/j.dsp.2015.11.001], URL: <https://doi.org/10.1016/j.dsp.2015.11.001>
2. Mrinmoy Bhattacharjee, S. R. M. Prasanna, Prithwijit Guha. "Time-Frequency Audio Features for Speech-Music Classification." *arXiv preprint* arXiv:1811.01222, 2018. URL: <https://arxiv.org/abs/1811.01222>
3. Toni Hirvonen. "Speech/Music Classification of Short Audio Segments." In: *2014 IEEE International Symposium on Multimedia (ISM)*, pp. 36–39, 2014. DOI: [10.1109/ISM.2014.18], URL: <https://doi.org/10.1109/ISM.2014.18>

Requirements for the semestral defence:

- Objektívni vyhodnocení (benchmark) stávajících metod pro klasifikaci řeči a hudby
- První verze klasifikátoru řeč/hudba natrénovaná na malé databázi

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Malenovský Vladimír, Ing., Ph.D.**
Head of Department: Černocký Jan, prof. Dr. Ing.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 7.5.2026

Abstract

This bachelor's thesis focuses on the problem of speech and music classification under real-time and causality constraints. The standard 2-class speech-vs-music formulation of the reference methods is extended with a third class for silence and ambient noise. The main objective is to design and implement a classifier satisfying these constraints and to objectively evaluate it against existing reference methods on a sufficiently diverse dataset. The first part of the thesis provides an overview of current approaches to audio signal classification, covering both traditional machine learning methods based on handcrafted features and modern neural network-based approaches. The second part focuses on the design and implementation of the dataset construction pipeline used to assemble the dataset for training and evaluation. The third part presents the experimental work. Four reference methods are implemented and compared using a benchmark on the constructed dataset. The subsequent experiments focus on the best-performing Temporal Convolutional Network (TCN). Based on these experiments, the proposed method called TCN-S is derived. The reference and proposed methods are then compared and analyzed. The results show that TCN-S achieves performance comparable to the reference TCN architecture while using only a fraction of the parameters and exhibiting lower computational latency.

Abstrakt

Táto bakalárska práca sa zaoberá kauzálnou klasifikáciou reči a hudby v reálnom čase. Štandardná formulácia referenčných metód, ktorá klasifikuje 2 triedy (reč vs. hudba) je rozšírená o tretiu triedu pre ticho a okolitý šum. Hlavným cieľom práce je navrhnúť a implementovať klasifikátor spĺňajúci tieto požiadavky a objektívne ho porovnať s referenčnými existujúcimi riešeniami na dostatočne rôznorodom datasete. Prvá časť práce zahŕňa prehľad súčasných prístupov využívaných v oblasti klasifikácie audio signálov, pričom pozornosť je venovaná tradičným metódam založeným na ručne extrahovaných príznakoch, ako aj moderným prístupom využívajúcim neurónové siete. Druhá časť je zameraná na návrh programu na tvorbu datasetov, ktorý bol použitý na vytvorenie datasetu využitého pri tréningu a evaluácii v tejto práci. Tretia časť demonštruje vykonanú experimentálnu prácu. Štyri referenčné metódy sú implementované a porovnané pomocou benchmarku na vytvorenom datasete. Nasledujúce experimenty sa zameriavajú na najefektívnejšiu temporálnu konvolučnú sieť (TCN). Na základe týchto experimentov je odvodená vlastná metóda nazvaná TCN-S. Referenčné a navrhnuté metódy sú následne porovnané a analyzované. Výsledky ukazujú, že TCN-S dosahuje porovnateľné výsledky ako referenčná TCN pri zlomku počtu parametrov a nižšom výpočtovom oneskorení.

Keywords

speech and music classification, on-line audio classification, causal Temporal Convolutional Network (TCN), deep learning, machine learning, dataset construction

Kľúčové slová

klasifikácia reči a hudby, on-line klasifikácia zvuku, kauzálna temporálna konvolučná sieť (TCN), hlboké učenie, strojové učenie, tvorba datasetu

Reference

HRIC, Marek. *Speech/Music Classification*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Vladimír Malenovský, Ph.D.

Rozšírený abstrakt

Klasifikácia reči a hudby je úloha, pri ktorej algoritmus pre každý časový rámec audio signálu rozhodne, či ide o reč alebo hudbu. Táto práca rozširuje úlohu o tretiu triedu reprezentujúcu zvuky pozadia (ticho a okolitý šum). Práca sa zameriava na on-line variant tejto úlohy, ktorý vyžaduje dostupnosť rozhodnutí v reálnom čase s minimálnou latenciou. Okrem tejto požiadavky je zavedené aj obmedzenie striktnej kauzality, t. j. klasifikátor využíva iba minulé vzorky signálu. Hlavným cieľom práce je navrhnúť a implementovať takýto klasifikátor a objektívne ho porovnať s existujúcimi referenčnými riešeniami na dostatočne rôznorodom datasete.

Najprv práca prezentuje prehľad súčasných prístupov využívaných v oblasti klasifikácie audio signálov, pričom pozornosť je venovaná tradičným metódam založeným na ručne extrahovaných príznakoch, ako aj moderným prístupom využívajúcim hlboké neurónové siete. Z tradičných metód sú predstavené rozhodovací strom od Lavnera a Ruinskiyho [23], model gaussovských zmesí a stroj podporných vektorov od Khonglaha a Prasannu [21], ktorí tieto modely navrhli na rovnakom extraktore príznakov. Ako referenčný neurónový model bola vybraná temporálna konvolučná sieť (TCN) navrhnutá Lemairom a Holzapfelom [25].

V ďalšej časti sa práca zameriava na návrh programu na tvorbu datasetov. Po zozbieraní štyroch verejne dostupných zdrojov bol vytvorený približne 100-hodinový dataset, využívaný pri tréningu a evaluácii referenčných aj experimentálnych modelov.

Hlavnú časť práce tvorí experimentálny postup vedúci k návrhu vlastnej metódy. Prvým vykonaným benchmarkom bolo porovnanie referenčných riešení, pričom referenčná TCN dosiahla macro F1 skóre 0,975, čo je o 0,1 viac ako najlepšie hodnotený tradičný model SVM. Na základe tohto benchmarku bola referenčná TCN vybraná ako hlavná referencia pre ďalšie experimenty.

Prvá experimentálna fáza sa začala ablačnou analýzou a analýzou senzitivity referenčného modelu TCN, ktorá ukázala, že snaha o zlepšenie presnosti ladením hyperparametrov je neefektívna. Ďalšie experimenty zahŕňali zmenu reprezentácie vstupného spektrogramu, pridanie neurónovej siete pred hlavnú TCN, výmenu TCN základu za rekurentnú alebo pozornosťnú architektúru, kombináciu predchádzajúcich experimentov a pridanie malej neurónovej siete za výstup TCN. Tieto experimenty viedli k modelu pomenovanému TCN-L, ktorý kombinuje príznaky Δ^2 na vstupe, 1-D konvolučnú sieť ako preprocesor a LSTM hlavu za TCN základom. TCN-L priniesol zlepšenie macro F1 skóre o takmer 0,01, avšak za cenu takmer $12\times$ väčšieho počtu parametrov. To motivovalo zmenu cieľa ďalších experimentov.

Druhá fáza experimentov sa zamerala na návrh modelu s menším počtom parametrov a nižším výpočtovým oneskorením, no porovnateľnou presnosťou vzhľadom na referenčnú TCN. Postupná redukcia šírky a hĺbky TCN architektúry viedla k modelu pomenovanému TCN-S, ktorý dosahuje rozdiel iba 0,003 macro F1 skóre oproti referenčnej TCN pri sedminovom počte parametrov a približne $21\times$ nižšom počte streamových MAC operácií na rámec.

Okrem presnosti boli modely vyhodnotené aj z hľadiska výpočtovej efektivity. Všetky modely dosahujú viac než dvojnásobnú rýchlosť oproti reálnemu času na jednovláknovom CPU, pričom najrýchlejší je model TCN-S. Výsledky potvrdzujú vhodnosť kauzálnych konvolučných architektúr pre on-line spracovanie audio signálu.

Hlavnými prínosmi práce sú návrh odľahčenej kauzálnej architektúry TCN-S pre trojtriednu on-line klasifikáciu audio signálu a návrh deterministickej pipeline na tvorbu datasetu, ktorá umožňuje rozšírenie o nové dátové zdroje bez potreby zásadných zmien.

Speech/Music Classification

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Vladimír Malenovský, Ph.D. I have listed all the literary sources, publications, and other sources, which were used during the preparation of this thesis. Claude and Claude Code were used for grammar checking, coding assistance, and proofreading during the work on this thesis.

.....
Marek Hric
May 13, 2026

Acknowledgements

I would like to thank my supervisor, Ing. Vladimír Malenovský, Ph.D., for his professional guidance, expertise, and feedback throughout the work on this thesis. I also thank my friends, family, and beer manufacturers for the support and encouragement.

Contents

1	Introduction	3
2	Background and problem context	5
2.1	Origins and applications	5
2.2	Characteristics of speech and music signals	5
2.3	On-line classification	6
3	Classification approaches	8
3.1	Overview	8
3.2	Output classes	9
3.3	Traditional methods	9
3.4	Deep learning methods	12
3.5	Streaming inference	17
4	Dataset construction	19
4.1	Mapping audio signals to output classes	19
4.2	Dataset design	20
4.3	Dataset construction methodology	21
4.4	Realized composition	22
4.5	Critical subset	23
5	Experiments and analyses	25
5.1	Evaluation methodology	25
5.2	Benchmark on reference models	26
5.3	Improving the TCN	27
5.4	Proposed lightweight variant	36
5.5	Analyses	37
6	Conclusion	45
6.1	Summary	45
6.2	Contributions	45
6.3	Limitations	46
6.4	Follow-up work	46
	Bibliography	48
A	Dataset sources	52

List of Figures

3.1	Example diagram of a generic speech/music classifier.	8
3.2	Causal dilated convolutions.	14
3.3	TCN residual block.	15
3.4	Receptive field with two convolutions per block.	16
3.5	TCN backbone. F represents the filter count in each block.	16
4.1	Dataset construction pipeline.	21
4.2	Realized composition of the full subset.	23
5.1	Per-class F1 (bars) and macro F1 score ($\diamond$) benchmark results.	27
5.2	Component substitution scan.	28
5.3	Hyperparameter sensitivity scan.	29
5.4	Spectral frontend scan.	31
5.5	Learned-preprocessor scan.	32
5.6	Backbone architecture scan.	33
5.7	Combined modifications.	34
5.8	Temporal head scan on the F+P TCN base.	35
5.9	Experimental path from reference TCN to TCN-S.	36
5.10	Cost vs effectiveness trade-off.	39
5.11	Parameter count against streaming MACs (log-log).	41
5.12	Classifier decisions on a pop music clip (top) and a clean speech recording (bottom).	42
5.13	TCN-S on a correctly classified <i>Background</i> clip (top) and the same clip with <i>Speech</i> overlaid (bottom).	42
5.14	Misclassified <i>Background</i> clip across all six classifiers (top) and the same clip with <i>Speech</i> overlaid on TCN-S (bottom).	43
5.15	Classification decisions on extreme music.	43
5.16	TCN-S decisions on speech over music (top), multi-speaker recording (middle), and multi-speaker over music (bottom).	44

Chapter 1

Introduction

Classification of speech and music involves determining whether an audio segment is speech, music, or neither. The focus of this work is on-line classification, the process of classifying a continuous live stream of data [13]. In contrast to the off-line approach, data are processed while being recorded, and the result is returned immediately after the computation is complete.

This task is typically a crucial preprocessing step in larger pipelines, such as automatic speech recognition [3], telephony, or speech and audio codecs that switch compression modes based on the input content [28]. The aim is therefore not only to improve the precision of the classifier but also to keep it computationally efficient. Complexity constraints have led researchers to develop relatively simple machine learning models, such as Decision Trees [23], Gaussian Mixture Models, or Support Vector Machines [21].

The improvement in computational power of devices that utilize applications requiring speech/music classifiers, combined with the rise of deep learning, has led to a resurgence and advancement in the field of speech and music classification in recent years. Researchers have experimented with several neural-network families for this task, including convolutional, recurrent, and convolutional recurrent architectures [7], with recent work reporting strong results from causal Temporal Convolutional Networks on broadcast audio [25].

Many existing solutions for on-line speech and music classification are either proprietary or limited in scope. Common limitations include restrictive assumptions about the input signal, such as the presence of only speech or only music, and a lack of training on more complex acoustic scenarios, for example overlapping speakers or mixed audio content. Other approaches are overly general and therefore inefficient or impractical when deployed specifically for this task.

The objective of this thesis is to design and implement a real-time, causal speech and music classifier that extends the 2-class formulation of the reference papers to a 3-class output space (*Speech*, *Music*, *Background*). A reproducible dataset construction pipeline is developed to assemble the training dataset from public sources, and four reference methods are implemented as baselines. The experimental program then proceeds in two phases: an effectiveness phase that explores the TCN’s design space, and a cost phase that derives **TCN-S**, a lightweight variant matching the strongest reference at a fraction of its computational cost.

The thesis is organized as follows. Chapter 2 describes the origins of the task, the characteristics of speech and music signals, and defines on-line classification with its constraints. Chapter 3 surveys existing approaches and presents the four reference models implemented in this thesis, along with the adjustments made for the 3-class on-line setting.

Chapter 4 covers the dataset construction pipeline, the resulting training dataset, and the manually labeled critical subset used for stress testing. Chapter 5 describes the evaluation methodology, benchmarks the reference models, presents the experimental program over the TCN’s design space, derives the proposed lightweight variant TCN-S, and reports cross-model analyses. Chapter 6 summarizes the contributions, limitations, and directions for follow-up work.

Chapter 2

Background and problem context

This chapter describes how the task of speech and music classification was first introduced to the public and how it evolved into a foundational building block of modern audio processing pipelines, summarizes the physical and acoustic characteristics that separate the two signals, and concludes with the definition and constraints of the on-line classifier.

2.1 Origins and applications

Automatic classification of speech and music originated in radio broadcasting, where it enabled receivers to automatically skip past speech and commercials in favor of music [36].

The growth of speech and music codecs introduced a parallel need for content classifiers at the codec front end: speech and music are best compressed by different codec families, and a misclassification at the input degrades compression quality [37].

Today, speech and music classification is a fundamental component of modern audio processing systems. It is used in voice-activated assistants, music streaming platforms, and telecommunication systems, where accurate content identification enables appropriate codec selection and efficient resource usage [28]. Beyond consumer applications, it also supports large-scale infrastructures such as broadcast monitoring and audio archiving.

2.2 Characteristics of speech and music signals

Speech differs from music along several physical and acoustic dimensions that every classifier in this work exploits. The differences arise from how each signal is produced and shape the time-frequency patterns visible in their spectrograms. The remainder of this section summarizes the properties that motivate the features and architectures introduced in Chapter 3.

Speech is produced by a vocal tract whose shape changes slowly between voiced and unvoiced sound units, and the glottal source is quasi-periodic with a pitch span of roughly three octaves [21]. Music is produced by instruments whose tones span up to six octaves, decay slowly, and fall on a fixed set of frequencies set by the instrument’s construction [4]. The alternation of vowel and consonant in speech also produces large frame-to-frame energy variation that does not appear in sustained musical tones.

These production differences imprint distinctive patterns on the log-mel spectrogram. Speech shows smooth arc-like striations whose harmonics shift continuously between sound units, while music shows long horizontal segments whose harmonics remain stationary be-

fore sharp transitions [4]. Speech also concentrates modulation energy near the syllabic rate of about 4 Hz, a pattern that is largely absent in instrumental music [21]. Hand-engineered features and learned spectrogram representations both target these regularities.

2.3 On-line classification

This thesis will use the definitions of the term “on-line processing” according to Eyben’s doctoral thesis [13]. On-line processing implicitly requires “real-time processing”, so both terms are introduced.

“Real-time processing refers to processing of data in real-time. This implies two things: (a) the processing of the data requires less time than the duration of the data, i.e., the time it will take to record or play-back the data at normal speed, and (b) the processing lag is as low as possible, ideally zero.” [13, p. 139]

“On-line processing refers to processing of a continuous live stream of data while data are recorded or transmitted, i.e., computing and returning results continuously before the end of the data stream has been seen.” [13, pp. 139–140]

With these definitions, on-line classification can be described as a process in which incoming data are processed continuously and classified into predefined classes, while maintaining low algorithmic delay. While the definition allows small lookahead into future samples, I have decided to keep the models in this thesis strictly causal.

In practice, on-line processing usually requires the use of a buffer to aggregate incoming samples. Once the buffer is filled, the classification is performed on the aggregated segment. This aggregation introduces an **algorithmic delay** between data acquisition and the availability of a classification result: the minimum span of audio that must be received before the algorithm can produce any output. For a causal system that operates on a fixed-length analysis frame (the unit of audio the algorithm processes to produce one decision), this cold-start delay equals the frame length and is independent of compute speed. It would persist on an arbitrarily fast machine. After the first frame, each subsequent decision waits only for one hop of new audio, since the prior analysis context is already buffered. The steady-state algorithmic delay is therefore the hop length, typically in the range of 10–30 ms, while the cold-start delay equals the frame length and is incurred once at stream start. Features that aggregate statistics over a longer temporal context may rely on an additional rolling buffer of past samples. This buffer carries prior context that has already been received and therefore does not contribute to algorithmic delay. It is a context window, not a waiting period.

Algorithmic delay is only part of the picture. A second contribution to the overall **latency** of an on-line classification system is the **computational delay**: the wall-clock time (the elapsed time on a real-world clock during execution) the implementation takes to extract features and produce a class label once an analysis frame is available. Computational delay is hardware-dependent and grows with the complexity of the feature extractor and classifier. Other contributions (audio acquisition, output dispatch, scheduling) are negligible compared to algorithmic and computational delay and are folded into the computational-delay budget for the rest of this thesis. Total latency is therefore **algorithmic delay** + **computational delay**, with the first term fixed by the model definition and the second measured per model in Section 5.5.1. On-line operation requires both to be small: algorithmic delay through a bounded analysis-frame duration, computational delay through computationally efficient algorithms with predictable execution times.

2.3.1 Constraints

Based on the above considerations, the following constraints apply to the on-line classification approach used in this work:

- **Causality:** The classification must be performed using only past and present samples, without access to future data.
- **Real-time operation:** All processing steps must complete within the time duration of the processed data segment.
- **Low algorithmic delay:** The hop between consecutive decisions imposed by the algorithm must be kept short, since it lower-bounds the steady-state response time of the system.
- **Low computational delay:** Feature extraction and classification must have predictable and sufficiently low wall-clock cost, so that decisions are produced faster than new hops of audio arrive.

Chapter 3

Classification approaches

The following chapter surveys existing approaches to speech and music classification and presents the models selected as references for this work, including their published trade-offs, the adjustments made to fit the 3-class on-line setting, and an analysis of their streaming behavior.

3.1 Overview

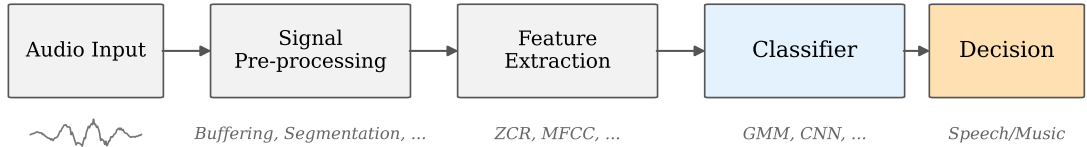


Figure 3.1: Example diagram of a generic speech/music classifier.

Classifiers are algorithms whose goal is to assign an input sample to one of a set of predefined classes. A minimal processing pipeline of a generic classification system is shown in Figure 3.1. Its first stage consists of pre-processing steps whose details depend on the architecture and application of the specific classifier. For on-line audio classification, this stage buffers and segments the input signal under the constraints described in Section 2.3.1.

Raw input samples do not provide a robust representation of an audio signal, which motivates the use of feature extraction in the majority of audio classification pipelines. These pipelines are themselves typically embedded as pre-processing modules in larger systems, where the classifier prevents subsequent stages from operating on unsuitable input.

Published speech and music classifiers fall into three broad lines: hand-engineered acoustic features paired with a traditional learner, learned features paired with a linear classifier, and end-to-end neural models. The three implemented references in this work cover the first and the third of these lines, and the remainder of this section sketches the wider landscape they sit in.

The hand-engineered feature line begins with the radio-monitoring discriminators of Saunders [36] and Scheirer and Slaney [37], who combined statistical measures of zero-

crossing rate, spectral centroid, spectral flux, and modulation energy with simple Gaussian classifiers. Lavner and Ruinskiy [23] produced a decision-tree-based system with a hand-engineered three-stage rule-based sieve over time- and frequency-domain statistics. Khonglah and Prasanna [21] pushed this line further with speech-specific features (excitation source, vocal tract, modulation spectrum) and a Gaussian Mixture Model (GMM) or Support Vector Machine (SVM) classifier. Bhattacharjee, Prasanna, and Guha [4] treated spectral peak locations across frames as time series and computed periodicity, zero-crossing rate, and second-order statistics on these sequences with a support-vector classifier.

The learned-representation line begins with Hirvonen [18], who learned a sparse-coding dictionary from log-mel spectrograms and fed the encoded segments to a linear support-vector classifier. End-to-end neural models replaced the dictionary with a learned convolutional or recurrent backbone, and Lemaire and Holzapfel [25] showed that a causal Temporal Convolutional Network outperforms recurrent references on broadcast audio while remaining feed-forward and parallelizable. The published trade-offs of the three implemented references are revisited alongside their adaptations in Sections 3.3 and 3.4.

3.2 Output classes

Every classifier in this thesis assigns each frame to one of three classes:

- **Speech:** human speech, even with background music or other speakers.
- **Music:** instrumental, vocal (including a cappella), or unconventional music content.
- **Background:** non-speech, non-music audio, including silence, ambient sound, and noise.

This formulation departs from the 2-class formulation of the reference papers [23, 21, 25] (introduced in the following sections). The other mentioned but non-implemented papers also do not address it [4, 18].

The third class makes the classifier deployable on its own. Codecs see silence and ambient noise alongside speech and music. A speech-vs-music-only model has no defined behavior on those inputs, so a codec deployment would otherwise need a separate activity gate to filter them out before classification. Folding the *Background* class into the classifier itself absorbs that decision, removing the need for a separate activity gate.

Which audio signals fall into each class is detailed in Section 4.1.

3.3 Traditional methods

Traditional methods of speech and music classification refer to the use of machine learning algorithms such as GMM, a decision tree, or SVM. Compared to deep learning, they offer low computational complexity at inference and rely on a hand-engineered feature set. Feature selection is required to keep the feature space compact while preserving its discriminative power. In practice this is done either through exhaustive experimentation or with automated selection algorithms, ranging from filter methods such as the analysis-of-variance (ANOVA) F-test [15] to wrapper methods such as Sequential Floating Forward Selection [33].

The following models were implemented as reference non-neural solutions for the speech and music classification problem. All traditional methods are implemented with the Python

library `scikit-learn` [31]. Feature extraction is implemented using NumPy [17], SciPy [41], or librosa [27] primitives.

All three models were originally designed as binary classifiers and were adapted to the 3-class task.

3.3.1 Decision tree

Original method. Lavner and Ruinskiy [23] proposed a hand-engineered three-stage sieve over a pool of nineteen time- and frequency-domain features. The features are computed on 40 ms frames with 50% overlap and aggregated per feature across 4 s segments at 100 ms hop via mean, standard deviation, and the mean and standard deviation of the frame-by-frame difference.

On the Scheirer-Slaney dataset [37] the authors report 99.07 % accuracy at low compute cost suitable for real-time operation, evaluated on a much larger test dataset than most prior work. Reported limitations are accuracy drops to roughly 93 % on electronic and pop music and a transition delay of approximately 6 s that follows from the 4 s segment window.

The features used in the original work are: Short-Time Energy, Zero-Crossing Rate, Band Energy Ratio, Autocorrelation Coefficient, 10 Mel-Frequency Cepstrum Coefficients (MFCCs), the Euclidean norm of the MFCC difference vector, Spectral Rolloff Point, Spectral Centroid, Spectral Flux, and Spectral Spread.

A non-parametric class-conditional probability density function is estimated for every feature, and five thresholds are derived per feature: two extreme thresholds (zero training-error per class), two high-probability thresholds, and one separation threshold. A separate feature subset is selected for each threshold type, using inclusion-fraction ranking, separation-power scoring, and floating selection over a scatter-matrix criterion.

At inference, each 4 s segment receives a grade in $[-1, +1]$ from how many of its features surpass each threshold. The grade is then smoothed by an exponentially-decaying weighted average of past segment decisions and discretized against an adaptive boundary.

Implementation. I replaced the sieve with a single `scikit-learn DecisionTreeClassifier`, preceded by a `StandardScaler` and an ANOVA F-test `SelectKBest` that picks the ten most discriminative components in a single step instead of running the paper’s per-threshold multi-stage scheme, simplifying the implementation and aiding extensibility.

On-line adaptation. To account for the on-line constraint of the task at hand, I reduced the frames from 40 ms to 20 ms, preserving the 50% overlap, and replaced the 4 s segments with a 300 ms rolling buffer of features over which the statistical aggregates are computed.

I kept the paper’s exponentially-decaying decision smoothing (forgetting factor of 0.9 over a 30-deep deque of post-classifier probabilities) but operate it per frame at 10 ms intervals. I dropped the adaptive boundary and the paper’s four-level output mode, since the learned tree predicts directly via $\arg \max$ over its class probabilities. These changes let the classifier predict each frame’s class every 10 ms, comfortably satisfying the on-line constraint.

Three-class extension. The paper trains only on speech and music, explicitly excluding silent and noisy frames. For the 3-class task, I gave background frames their own training labels and let the tree fit all three classes simultaneously. The smoothed $\arg \max$ extends naturally over {speech, music, background} without further changes.

3.3.2 Gaussian mixture model and support vector machine

Original method. Khonglah and Prasanna [21] proposed a Gaussian mixture model and a support vector machine sharing the same feature extraction pipeline. The authors report the SVM reaching 95.87 % classification accuracy on the Scheirer-Slaney dataset [37], and the speech-specific features generalize across the Scheirer-Slaney, GTZAN [40], and Indian Broadcast News datasets with low mutual correlation between them. Reported limitations are the threshold variant’s reliance on hand-tuning and the work’s restriction to non-vocal music.

Features. The original work specifies 30 ms frames with 1 ms hop for speech-specific features. Frame size for the existing features is not specified.

Speech-specific features: Normalized Autocorrelation Peak Strength (NAPS) of the Zero Frequency Filtered Signal, Peak-to-Sidelobe Ratio (PSR) of the Hilbert Envelope of Linear Prediction (LP) Residual, Log-Mel Spectrum Energy, and Modulation-Spectrum Energy at 4 Hz. Existing features: Zero-Crossing Rate, Spectral Centroid, Spectral Flux, and Spectral Rolloff Point.

The extracted features are then accumulated over 1 s non-overlapping windows. Long-term statistics are computed and passed to the classifiers: the mean and variance, plus the percentage of low-energy frames (frames whose total energy is below one-third of the average energy in the window).

On-line adaptation. To run the feature-extraction pipeline on-line, I kept the original 30 ms frame length and 1 ms hop for NAPS and log-mel, and vectorized their computation. Since the Levinson-Durbin LP analysis [26] is sequential by nature, I sped up its calculation by reducing its cadence to 100 Hz. A hop-length scan confirmed that PSR remains > 0.99 correlated with the 1 ms reference up to a 10 ms hop.

For the existing features, for which the paper does not specify a hop, I use 30 ms frames at 15 ms hop to match the streaming output rate. Finally, I replaced the 1 s non-overlapping statistics window with a rolling buffer updated every 15 ms.

Khonglah and Prasanna’s work does not specify any feature normalization. I added it to both classifiers for reasons described in the following subsections.

Gaussian mixture model

The reference GMM uses two models for the speech and music classes. I added a third model to accommodate the *Background* class. Each of the models has eight mixture components with diagonal covariance matrices.

The GMM’s sensitivity to outliers motivates feature normalization using `RobustScaler`, which scales by their per-feature median and interquartile range. I then clamp the scaled features to ± 10 to prevent residual outliers from triggering the kind of numerical-instability failures known to affect GMM fitting [5, §9.2.2]. Without the clamp, training produced NaN values and aborted.

Frames were originally classified by the sigmoid-mapped log-likelihood delta against a learned threshold. At the 15 ms frame hop, per-frame predictions flicker between classes because each frame’s feature aggregates are computed over slightly different rolling windows.

Averaging the classifier outputs over a short buffer damps this flicker. Concretely, I average the per-class log-likelihoods over a 66-frame rolling buffer (≈ 1 s at 15 ms hop). I replaced sigmoid mapping with $\arg \max$ over the smoothed log-likelihood vector.

Support vector machine

The reference describes SVM with regularization parameter $C = 1$ and radial basis function (RBF) kernel with $\gamma = 3$. I kept these hyperparameters identical and implemented the SVM using scikit-learn’s `SVC`. No adaptation from 2 to 3 classes was needed, because `SVC` supports multiclass classification by default.

The RBF kernel is dominated by pairwise Euclidean distances between feature vectors, so feature magnitudes need to be on the same scale. A plain `StandardScaler` (z-score normalization [16]) is sufficient here.

The reference trained the SVM on 160 fifteen-second recordings, yielding 2,400 1 s non-overlapping windows, and produced a binary classification per window. The RBF kernel generally scales between $O(n^2)$ and $O(n^3)$ with training set size.

Training on the full training set, which produces over 20 million 15 ms frames, is infeasible. To make training manageable, I reduced the training set to 50,000 frames per class. This subset of 150,000 frames is still roughly $60\times$ bigger than the training set used in Khonglah and Prasanna’s work.

The reference SVM produced one decision per non-overlapping 1 s window with no further smoothing. I average the 3-element decision-function vectors over a 20-frame rolling buffer (≈ 300 ms) to stabilize the per-frame decisions at the 15 ms streaming hop. Compared to the GMM, the SVM smoothing buffer is shorter (20 vs 66 frames) because SVM’s outputs are more stable and in a comparable range across frames. Finally, the class of the frame is determined by $\arg \max$ over the smoothed vector.

3.4 Deep learning methods

Deep learning models eliminate the hand-engineered feature step that constrains the traditional classifiers in Section 3.3. Given a normalized log-mel spectrogram, the network learns its own internal representations and the decision boundary jointly, trading the interpretability of explicit features for capacity to capture patterns that hand-engineered statistics can miss.

Several families of neural architectures have been applied to audio classification. The earliest reference is the multi-layer perceptron [35], which processes each input frame independently and serves as the reference against which sequence-aware models are compared. The Convolutional Neural Network (CNN) [24], originally designed for image recognition, has been applied to spectrograms by treating them as two-dimensional images and learning local time-frequency patterns through stacked convolutions, producing strong results on tasks where the input window is fixed. The Recurrent Neural Network (RNN) [12] models temporal dependencies explicitly by carrying a hidden state forward through time, and the Long Short-Term Memory (LSTM) variant [19] solves the vanishing-gradient problem that limits vanilla RNNs.

3.4.1 Convolutional neural network

The CNN’s central operation is the convolution: a learned kernel slides across the input, computing a dot product at each position to produce a feature map. For a one-dimensional

input x and a learned kernel w of size K , the convolution at position t is

$$y[t] = \sum_{k=0}^{K-1} w[k] \cdot x[t - k] \quad (3.1)$$

the same expression extends to two dimensions by indexing over both the time and frequency axes of the spectrogram.

In practice, a convolutional layer learns many kernels in parallel. Each is called a **filter**, and the feature map it produces is one **channel** of the layer’s output. A conv layer is therefore characterized by two hyperparameters: the **kernel size** K (the spatial extent of each filter) and the **filter count** F (the number of output channels).

Two properties make this attractive for spectrogram inputs.

First, the kernel weights are shared across all positions, so the same time-frequency pattern is detected regardless of where it appears in the input.

Second, stacking convolutional layers builds a hierarchy: early layers respond to simple local patterns, and deeper layers compose them into longer-range structures. The set of input positions that influence a given output is called the network’s **receptive field**. It grows linearly with depth, by $K - 1$ positions per added layer.

A special case is the 1×1 convolution ($K = 1$), which acts as a learned linear combination of the input channels at each frame. It does not enlarge the receptive field but only mixes or re-projects channels.

3.4.2 Temporal convolutional network

Temporal Convolutional Networks (TCN) were introduced by Bai et al. [2] as a feed-forward alternative to recurrent sequence models. A TCN keeps the CNN’s parameter sharing and feature hierarchy but applies them along the time axis only, with one modification that adapts the architecture to long sequences: dilation.

Dilated convolutions read input samples at stride d instead of at consecutive positions, so the same K kernel weights (with K odd) span $(K - 1) \cdot d + 1$ input positions centered on t :

$$y[t] = \sum_{k=-(K-1)/2}^{(K-1)/2} w[k] \cdot x[t - k \cdot d] \quad (3.2)$$

Stacking L such convolutions at distinct dilations $d_1, \dots, d_L$ yields a receptive field of

$$\text{RF} = 1 + (K - 1) \sum_{l=1}^L d_l. \quad (3.3)$$

With exponentially-increasing dilation $d_l = 2^{l-1}$, this simplifies to

$$\text{RF} = 1 + (K - 1)(2^L - 1), \quad (3.4)$$

exponential in L .

Dilation lets the network access long-range temporal context without the rolling feature buffer or statistical aggregates that capture context over multi-frame windows.

Lemaire and Holzapfel [25] demonstrated this architecture on broadcast speech and music detection, where their non-causal extension reportedly reached F1 scores (explained in

Section 5.1.2) of 0.971 on music and 0.957 on speech, outperformed their picked references, and trained roughly $4\times$ faster. The non-causal kernel violates the on-line constraint defined in Section 2.3.1 because it reads future input, and its receptive field grows much larger than the training chunk length. The TCN’s feed-forward design also makes it parallelizable to train and bounded in memory at inference.

3.4.3 Causal TCN

The on-line constraint defined in Section 2.3.1 requires that each output frame depend only on past inputs. The causal TCN, presented in the same work by Lemaire and Holzapfel, enforces this by shifting the kernel of Equation 3.2 so it covers only past positions. The index k runs from 0 to $K - 1$ instead of being centered on zero, giving the causal dilated convolution

$$y[t] = \sum_{k=0}^{K-1} w[k] \cdot x[t - k \cdot d] \quad (3.5)$$

where every sampled input $x[t - k \cdot d]$ lies at or before t .

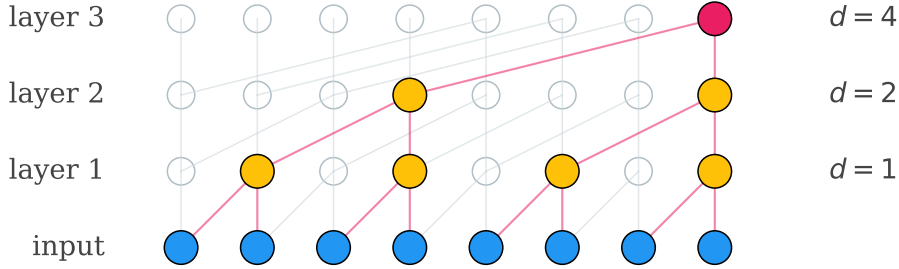


Figure 3.2: Causal dilated convolutions.

Figure 3.2 shows three such convolutions with kernel size 2 and dilations 1, 2, 4. The rightmost output neuron’s receptive field spans all 8 past input positions.

This causal variant of the TCN is adopted as the reference architecture for this work. All subsequent mentions of **TCN** refer to this causal variant.

Architecture

The TCN architecture builds from three nested levels: the residual block, the stack, and the backbone.

A **residual block** (Figure 3.3) is the smallest unit. Let c denote the number of convolutions per dilation level inside a block. A block contains c causal dilated convolutions that share the same dilation factor d and kernel size K , each followed by weight normalization, a non-linearity, and dropout. A skip connection adds the block’s input to the output of the second dropout, with a 1×1 convolution inserted on the residual path when the input and output channel counts differ. Bai et al. [2], Lemaire and Holzapfel [25] (via the keras-tn [34] implementation), and this work all set $c = 2$ and use ReLU as the non-linearity. A block thus replaces a single convolution as the per-dilation unit, so L from Equation 3.4 now counts blocks.

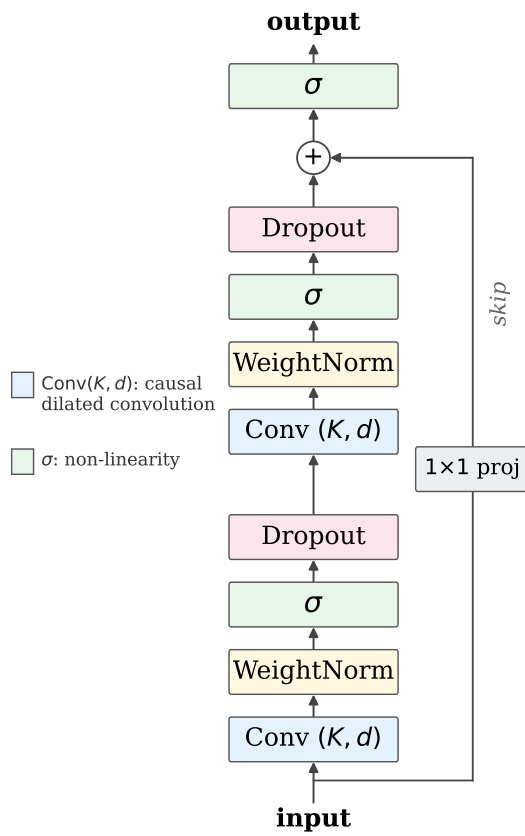


Figure 3.3: TCN residual block.

Each convolution extends the receptive field back by $(K - 1) \cdot d$ frames, so c such convolutions together extend it by $c(K - 1) \cdot d$. Equation 3.4 therefore generalizes to

$$\text{RF} = 1 + c(K - 1)(2^L - 1). \quad (3.6)$$

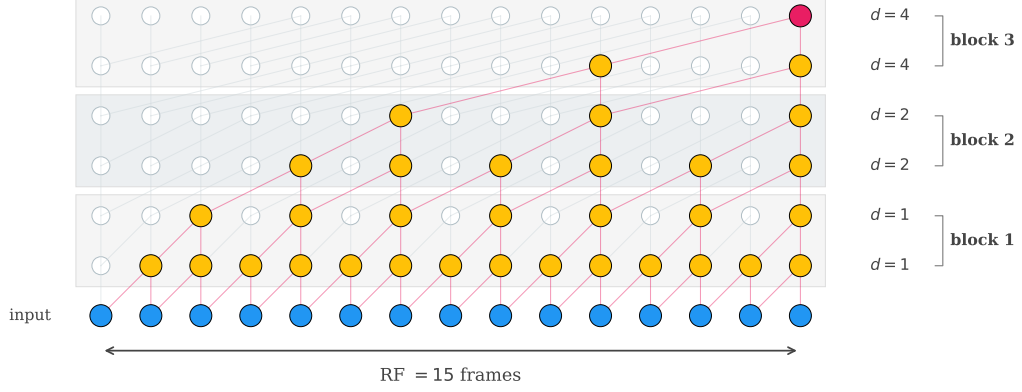


Figure 3.4: Receptive field with two convolutions per block.

Figure 3.4 shows the same kernel size and dilations as Figure 3.2 ($K = 2$, dilations 1, 2, 4) but with $c = 2$.

A **stack** groups L residual blocks with dilations $1, 2, 4, \dots, 2^{L-1}$. Stacking blocks with doubling dilation, rather than letting the dilation grow indefinitely, keeps the maximum dilation bounded while still multiplying the receptive field.

The **backbone** (Figure 3.5) chains S stacks together. The input log-mel spectrogram is first projected by a 1×1 convolution to a feature map whose channel count matches the residual blocks' filter count. A final 1×1 convolution at the end of the backbone maps the feature map to the three output classes, producing the per-frame logit tensor.

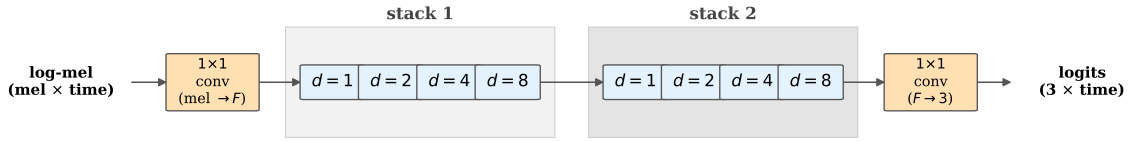


Figure 3.5: TCN backbone. F represents the filter count in each block.

Across S stacks, Equation 3.6 extends to

$$\text{RF} = 1 + cS(K - 1)(2^L - 1). \quad (3.7)$$

Implementation details and adaptations

The reference network operates on log-mel spectrograms. The audio is resampled to 22050Hz mono, a Short-Time Fourier Transform is computed with a Hann window of 1024 samples and a hop of 512 samples, and the squared magnitude is passed through 80

mel filters spaced between 27.5 Hz and 8 kHz. The resulting features are put on a logarithmic scale and normalized to zero mean and unit variance over the training set. Reference training uses fixed-length chunks of 270 frames at batch size 32, optimizes binary cross-entropy with stochastic gradient descent (SGD, momentum $m = 0.9$), divides the learning rate by ten when the validation loss has not improved for three epochs, and stops once five consecutive epochs show no improvement.

The hyperparameters adopted in this work fall within the ranges the paper allows. Both are listed in Table 3.1.

Parameter	Paper search space	This work
Number of filters	$\{8, 16, 32\}$	16
Number of blocks per stack	1–4	4
Number of stacks	3–10	3
Kernel size	$\{3, 5, 7, \dots, 19\}$	5
Dropout	0.05–0.5	0.5

Table 3.1: Reference TCN hyperparameter search space and used configuration.

The network has been implemented using the PyTorch [30] library. The paper’s recipe needs two adaptations for this setup.

First, the output layer projects to three classes (speech, music, and background) instead of two, producing a $(B, 3, T)$ logit tensor at the same frame rate as the input spectrogram. Here B is the batch size (32 during training, 1 during streaming inference) and T is the number of frames per forward pass (128 during training). Multi-label compatibility has been kept through per-class Binary Cross-Entropy (BCE). This way, no semantic change is needed to adapt the output from 2 to 3 classes. Implementation-wise, `BCEWithLogitsLoss` has been used, which is numerically equivalent to $\text{BCE} + \text{sigmoid}$ but applies the sigmoid in log-space for numerical stability. The final prediction is picked from the per-class probability vector using `arg max`.

Second, the chunk length is reduced from 270 frames to 128. At the original 270 frames ($\approx 6.3\text{s}$), 61% of the speech clips in the training set (described in Chapter 4) were too short to produce a single chunk, which biased the model toward music. 128 frames ($\approx 3\text{s}$) recovers the full distribution.

3.5 Streaming inference

The on-line constraint defined in Section 2.3.1 requires every classifier to produce a fresh prediction at the streaming output rate using only past audio.

The traditional models in Section 3.3 satisfy this by construction (their feature extractor is per-frame and the smoothing buffers accumulate decisions on the fly), so no further analysis is needed at the architectural level.

The TCN, in contrast, is a pure batch function from a mel chunk to per-frame logits, so its on-line behavior depends on the architecture itself. Causality of the dilated convolutions guarantees that the streaming output is bit-identical to the corresponding off-line forward pass on the same audio, with no padding, look-ahead, or boundary correction. This bit-identity is verified end-to-end by a parity check that feeds one second of fixed-seed random audio through both forward paths and requires the two resulting output tensors to match exactly, element-wise.

Both model families therefore meet the on-line constraint at the architectural level. Per-frame latency, real-time factor, and end-to-end frame timing are reported in Section 5.5.1.

Chapter 4

Dataset construction

This chapter describes the data used to train and evaluate every classifier in this thesis. Section 4.1 enumerates the kinds of audio signals and maps them onto the three output classes. Section 4.2 describes the dataset’s structure: subset sizes, subclasses, and the train, validation, and test split policy. Section 4.3 walks through the build pipeline that realizes the design. Section 4.4 reports the realized composition. Section 4.5 describes a separately constructed set of critical audio samples, a late addition for manual behavioral analysis of the models.

4.1 Mapping audio signals to output classes

The mapping of audio signals to the three output classes defined in Section 3.2 is grounded in the audio-signal categorization of Al-Shoshan [1, p. 97], which partitions audio into nine categories (paraphrased from the original):

1. Speech signal composed of a single talker within a specific time period.
2. Pure music signal without any speech component.
3. Mixture of single-talker speech and background music.
4. Songs, i.e., a mixture of music and a singer’s voice.
5. Singing without musical accompaniment.
6. Abnormal music, including unconventional cadences, human whistles, or non-musical sounds embedded as part of a musical structure.
7. Speech signal composed of two or more speakers talking simultaneously.
8. Non-speech and non-music signals, such as car, motor, or fan sounds.
9. Complex sound mixtures involving multiple speakers, singers, and musical sources.

The nine categories are projected onto the three output classes as follows. Categories 1, 3, and 7 (single-talker speech, speech with background music, multi-talker speech) map to *Speech*. Categories 2, 4, 5, 6, and 9 (pure music, songs, unaccompanied singing, atypical music, complex multi-source mixtures) map to *Music*. Category 8 and pure silence map to *Background*.

4.2 Dataset design

Section 4.2.1 enumerates the subsets, Section 4.2.2 the per-class subclass inventory, Section 4.2.3 the timestamped label format, and Section 4.2.4 the train, validation, and test split.

4.2.1 Subsets

The dataset is built in two training subsets plus the held-out evaluation subset. Table 4.1 lists their roles.

Subset	Size	Purpose
full	~ 6 000 min (100 h)	Canonical training and evaluation set.
critical	~ 75 min	Stress benchmark (Section 4.5).

Table 4.1: Dataset subsets, their sizes and roles.

The construction methodology in Section 4.3 produces two training subsets from the same public sources: the full subset used throughout this thesis, and a smaller mid subset of approximately 1 200 min (20 h) originally intended as a preview release. Licensing constraints on the sources prevent redistributing either subset, but both remain reproducible locally from the original public sources.

4.2.2 Subclasses

Each output class is split into subclasses that capture acoustically distinct conditions within the class. The four synthetic speech subclasses combine speech with noise, music, or a second speaker.

We abbreviate “speech over music” as SoM and “multi-speaker over music” as MSoM. Table 4.2 lists the inventory. Section 4.3.2 describes the mix policies.

Class	Subclasses
<i>Speech</i>	clean, dirty, noisy*, multi-speaker*, SoM*, MSoM*
<i>Music</i>	instrumental, electronic, pop, rock, hip-hop, folk, a cappella
<i>Background</i>	noise

Table 4.2: Subclass inventory across the three output classes. Asterisks mark synthetic subclasses generated at build time.

4.2.3 Frame-level labeling

Each clip carries a list of timestamped label intervals rather than a single clip-level tag. A label entry is a triple (*Class*, start, end), where the start/end timestamps are millisecond offsets into the clip with the interval treated as half-open. A clip’s full manifest is the concatenation of these intervals, covering the clip with no gaps.

This per-frame formulation is what makes the dataset usable for streaming evaluation (Section 5.1). A single clip can contain a transition between speech and music, or a stretch of background audio bridging two speech segments, and every frame in the clip has its own

ground-truth label. The same format is used uniformly across the dataset. Clean speech receives *Speech* intervals spanning the clip duration interleaved with *Background* intervals during the breaks in speech. The synthetic SoM augmentation interleaves *Speech* and *Music* intervals. Critical-subset clips carry as many intervals as the manual annotation requires.

4.2.4 Split policy

The full subset uses an 84%/8%/8% split for train, validation, and test, applied per source per subclass. This guarantees that every subclass appears in every partition even when a source contributes very few clips.

4.3 Dataset construction methodology

The build pipeline proceeds in three stages, summarized in Figure 4.1.

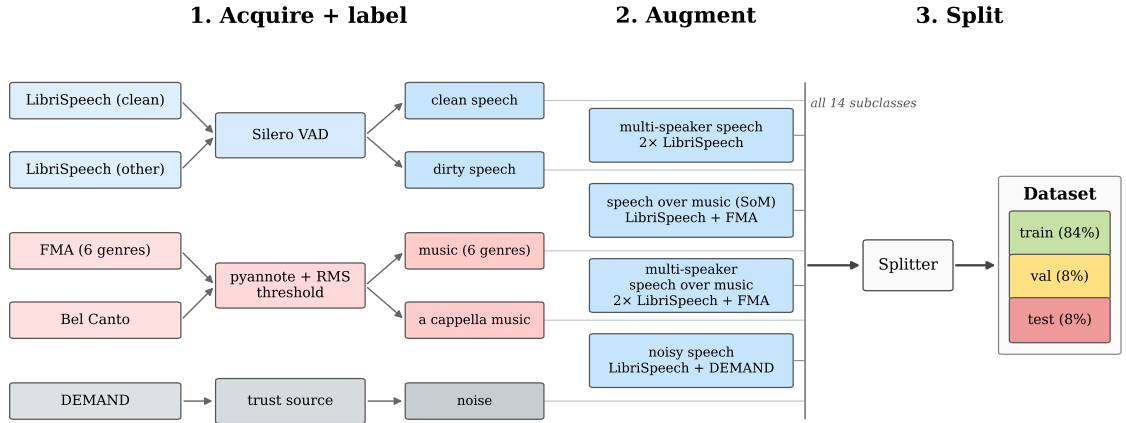


Figure 4.1: Dataset construction pipeline.

- **Stage 1** produces the real subclasses from external sources via class-specific labelers.
- **Stage 2** generates four synthetic speech subclasses by mixing real subclasses at build time.
- **Stage 3** interleaves the recordings into the three splits per source (Section 4.2.4).

4.3.1 Sources and automatic labeling

The complete enumeration of external sources is given in Appendix A. The pipeline groups them by output class and applies a class-specific labeler to each clip.

Speech. *Speech* material is drawn entirely from LibriSpeech ASR [29]. The clean training split supplies the clean speech subclass and the other training split supplies dirty speech. Each clip is processed by Silero VAD [38] to identify speech-active regions. Clips are split at silence boundaries and only the speech-active segments are retained.

Music. *Music* material combines the Free Music Archive (FMA) [9] with the Bel Canto a cappella dataset from CCMusic [43]. FMA contributes six genres in equal proportion: instrumental, electronic, pop, rock, hip-hop, and folk. Bel Canto contributes the seventh subclass, a cappella music, which is the music subclass closest to speech in feature space. Its inclusion forces the model to learn a clean speech-vs-vocal-music boundary at training time and exposes any remaining errors at evaluation. A custom music labeler combines a pyannote-audio voice-activity segmentation model [6, 32] with a root-mean-square (RMS)-based loudness threshold. The threshold is computed over 20 ms windows, with the per-clip noise floor set at the 20th percentile of RMS values scaled by 0.35. A frame is labeled music if the segmentation model identifies it as speech-active (instrumental music is similar enough to speech to register on this voice-activity model) or if its RMS exceeds the noise floor.

Background. Environmental noise is drawn from the DEMAND dataset [39], recordings spanning office, traffic, kitchen, and outdoor environments. Labeling is trivial: every clip is tagged *Background* as a whole.

4.3.2 Synthetic augmentations

Four synthetic subclasses fill in deployment-relevant conditions that no real-audio source covers cleanly. Each is generated deterministically at build time from the speech, music, and noise sources above, and carries the *Speech* class label. Table 4.3 summarizes the four constructions.

Subclass	Construction
multi-speaker	Two LibriSpeech speakers overlaid at -5 to $+5$ dB relative gain.
SoM	LibriSpeech with FMA music attenuated by 10 dB.
MSoM	Multi-speaker overlay placed over attenuated FMA music.
noisy	LibriSpeech with DEMAND noise at 5 to 20 dB SNR.

Table 4.3: Synthetic speech subclasses generated at build time.

Frame labels follow an overlay rule: the base clip’s *Speech* label applies throughout, and on its *Background* intervals the secondary clip’s labels take over. For SoM this gives *Speech* where the LibriSpeech speaker is active and *Music* during the silences. The same rule produces multi-speaker, MSoM, and noisy speech by analogy.

4.4 Realized composition

Figure 4.2 visualizes the realized composition of the full subset, broken down by subclass. The 40/40/20 class share matches the design ratio.

The center pie shows the top-level class split. The side pies break down speech and music into their subclasses, with each slice labeled in minutes and as a share of the parent class.

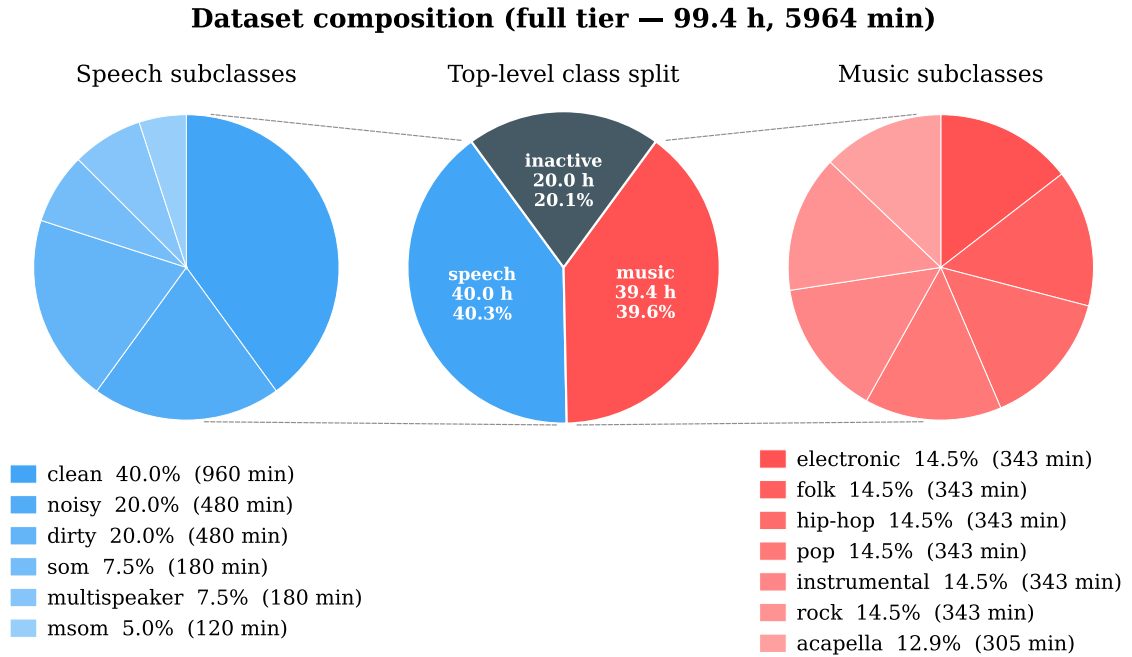


Figure 4.2: Realized composition of the full subset.

4.5 Critical subset

The critical subset is a small collection of 113 clips (~ 75 min total) built late in the work to sanity-check the models under known stress conditions. Clips are hand-selected by stress condition and labeled manually.

4.5.1 Methodology

The critical subset is built from 25 recordings hand-picked from publicly accessible YouTube material:

- 8 speech sources: six clean, one multi-speaker, one whispered.
- 10 music sources: seven in-distribution genres plus jumpup, speedcore, and beatbox as out-of-distribution stressors.
- 7 background-noise sources covering airport, birdsong, cafeteria, cooler fan, fireplace, rain, and street, each acting as a direct stressor clip and as mixing material for the noisy-speech generator.

Frame-level labels were produced manually rather than through automatic labelers, for ground-truth precision.

Two build-time routines expand the source pool into the 113-clip, ~ 75 min subset. A noisy-speech generator forms the Cartesian product of the six clean-speech sources, the seven noise sources, and two SNR levels (0 dB and -3 dB), producing 84 noisy-speech clips.

A speech-over-music (SoM) mixer overlays clean speech onto music at fixed gain offsets, producing 2 SoM and 2 MSoM clips from manually labeled sources.

Frame labels on the synthetic clips follow the same overlay rule as the full subset’s augmentations (Section 4.3.2).

4.5.2 Coverage

The critical subset mixes two groups of clips: stressors and in-distribution references.

Stressors are the focus of the subset, each one isolating a condition expected to challenge the classifiers. Table 4.4 summarizes the conditions exposed.

Stressor	Stress condition
airport, cafeteria	Crowd babble, voice-like background content.
cooler fan	Steady mechanical hum.
street	Outdoor mix: traffic, footsteps, distant voices.
birdsong	High-frequency harmonic non-speech sounds.
fireplace, rain	Ambient stationary noise.
whispered speech	Whisper, voiceless and low-energy.
noisy speech at 0 and -3 dB	Speech at extreme SNR.
extreme music	Extreme tempos and distortion.
beatbox music	Vocal-only music, no instrumental scaffolding.

Table 4.4: Stressors and the condition they expose in the critical subset.

In-distribution references repeat most standard subclasses for sanity comparison against a model’s behavior on familiar material. Dirty speech is left out since the noisy-speech augmentations already cover degraded speech.

Chapter 5

Experiments and analyses

In this chapter the experiments leading to the proposed solution of this thesis are presented.

First, the evaluation methodology is presented in Section 5.1. Then, the four implemented reference models are benchmarked in Section 5.2, followed by experimental analysis with the aim of improving the effectiveness of the best reference model in Section 5.3. Once it became clear that the reference model and its variants improved with diminishing returns on the test split, I changed direction. The new aim was to reduce parameter count and computational delay while keeping macro F1 score comparable with the reference TCN. This is presented in Section 5.4. Finally, the proposed models are analyzed in comparison to the references in Section 5.5.

5.1 Evaluation methodology

This section describes the frame-level evaluation procedure, the motivation behind the F1 metrics used, and the significance test applied to every classifier.

5.1.1 Frame-level evaluation

The on-line constraints (Section 2.3.1) require every classifier to operate at **frame level**, with the output rate matching each model’s frame hop (TCN 23.2 ms, GMM and SVM 15 ms, decision tree 10 ms). Each output prediction is compared against a per-frame ground-truth label. Each frame is labeled with the class that holds the largest share of its duration in the dataset’s timestamped intervals (Section 4.2.3).

5.1.2 Metrics

For each class $c \in \{Speech, Music, Background\}$, let TP_c , FP_c , and FN_c denote the number of true positive, false positive, and false negative frames over the evaluation split, computed one-vs-rest. Precision and recall for class c are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad (5.1)$$

and the **per-class F1** is their harmonic mean:

$$F1_c = \frac{2 P_c \cdot R_c}{P_c + R_c} = \frac{2 TP_c}{2 TP_c + FP_c + FN_c}. \quad (5.2)$$

F1 weights false positives and false negatives equally, so a model cannot trade one for the other to inflate the score.

The **macro F1 score** is the unweighted mean of the three per-class F1s:

$$\text{macro F1} = \frac{1}{3} \sum_c F1_c. \quad (5.3)$$

Averaging the per-class F1 gives every class equal weight regardless of frame count, so smaller classes contribute as much to the final score as larger ones. The macro F1 score is the headline metric for every experiment in this chapter.

Per-class F1 shows the contribution of each class to the macro F1 score.

Each reported macro F1 score is accompanied by a 95 % **confidence interval (CI)**: a band around the score that captures how much it could plausibly have shifted with a different mix of clips in the test split. The CI was added in response to a problem encountered in the first three experiments of Section 5.3 (Sections 5.3.1 to 5.3.3): many variants’ macro F1 scores landed within a few thousandths of the reference TCN, and a +0.003 gain could equally reflect a real improvement or just noise from the specific clips in the test split. The scores alone offered no way to tell.

The CI is constructed by a **cluster bootstrap** [11, 8]: clips, not individual frames, are the unit of resampling because frames within one clip share speaker, recording conditions, and buffer state. Frame-level resampling would treat each frame as an independent measurement and shrink the CI artificially. 1 000 alternative “test splits” are built by drawing clips from the actual test split at random with replacement, each one the same clip count as the original but with a different mix: each draw is independent, so the same clip can appear multiple times in a resample and roughly 37 % of the original clips are absent from it. The macro F1 score is recomputed on each, producing a distribution of 1 000 alternative scores around the actual test-split score. The CI is the range from the 2.5th to the 97.5th percentile of this distribution.

CIs are read as a guide rather than a strict cutoff. A CI that cleanly separates from the reference’s is taken as strong evidence of a real difference. Overlapping CIs are read more cautiously: the gap may still reflect a real effect, but the test split does not settle the question on its own.

The CI captures test-split variability only. Training itself is non-deterministic, since random weight initialization, data shuffling, and parallel arithmetic produce slightly different checkpoints from the same recipe on each run. A fixed random seed is used for every variant in this thesis to hold this second source of variance constant, and a multi-seed reproduction that would also capture training variance is listed as future work in Section 6.4.

5.2 Benchmark on reference models

The four reference classifiers are benchmarked on the test split (Section 4.2.1). The strongest of them will be the focus of the experiments that follow. Figure 5.1 presents the results. DT abbreviates the decision tree in plots and tables.

The TCN wins. It sits roughly 0.10 macro F1 score above the strongest traditional model. Its lead holds across all three classes.

The three traditional references cluster between 0.825 and 0.875 macro F1 score, while the TCN reaches 0.975, roughly twice that cluster width above the strongest traditional model. The ranking therefore does not depend on which traditional model is taken as the comparison reference.

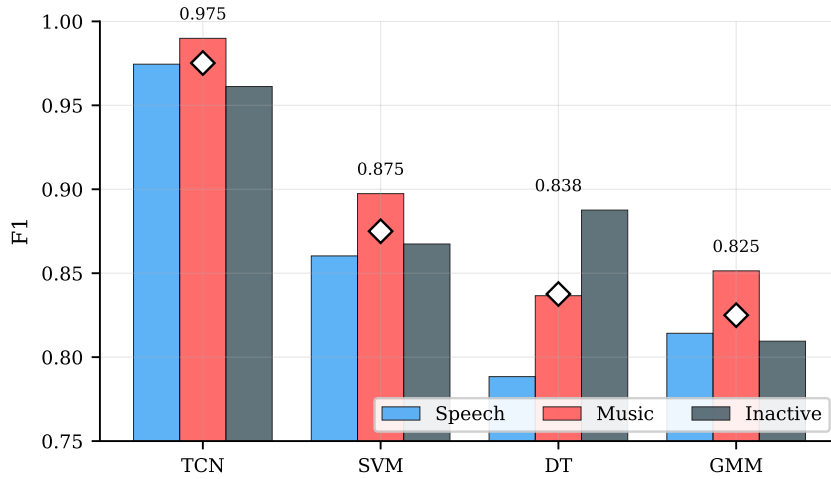


Figure 5.1: Per-class F1 (bars) and macro F1 score ($\diamond$) benchmark results.

The causal TCN is selected as the experimental focus for the rest of the chapter.

5.3 Improving the TCN

The reference TCN achieves macro F1 score of 0.9752 with a CI of [0.9687, 0.9794]. This section asks whether further improvements are available. The TCN is analyzed first, then five experiments replace or add to its components.

5.3.1 Sensitivity analysis and ablation

The reference configuration is summarized in Table 5.1.

Architecture		Optimization	
filters	16	optimizer	SGD, momentum 0.9
layers per stack	4	learning rate	10^{-3}
stacks	3	batch size	32
kernel size	5	loss	BCE-with-logits
dropout	0.5	Input	
normalization	WeightNorm	frontend	80-bin log-mel
activation	ReLU	chunk length	128 frames

Table 5.1: Reference TCN configuration.

Each experimental variant changes exactly one of these parameters, retrains for 50 epochs on the full subset with early-stopping patience of 5 epochs, and is compared against the reference TCN. The variants split into two groups:

1. **Structural and categorical variation** either removes a component (ablation) or swaps a discrete choice along one of four categorical dimensions: optimizer, normalization, activation, or loss.

2. **Hyperparameter sensitivity** varies a numerical parameter (learning rate, dropout, layer count, filter count, kernel size, sequence length, batch size).

Several groups landed within ± 0.0003 of the reference. Those are pruned from the active configuration and listed at the end of this section.

Structural and categorical variation

The only structural ablation tested removes the residual (skip) connections in every TCN block. Without them, macro F1 score collapses to 0.1884: the model predicts *Music* for every frame, with *Speech* and *Background* recalls both 0. The residual connections are therefore essential.

The substitution scan swaps each of three categorical choices in the reference TCN (optimizer, normalization, activation) for an alternative. Figure 5.2 reports the result. The same plot style is reused for every variant scan in the rest of Section 5.3: bars show macro F1 score with CI whiskers, and the gray band marks the reference CI.

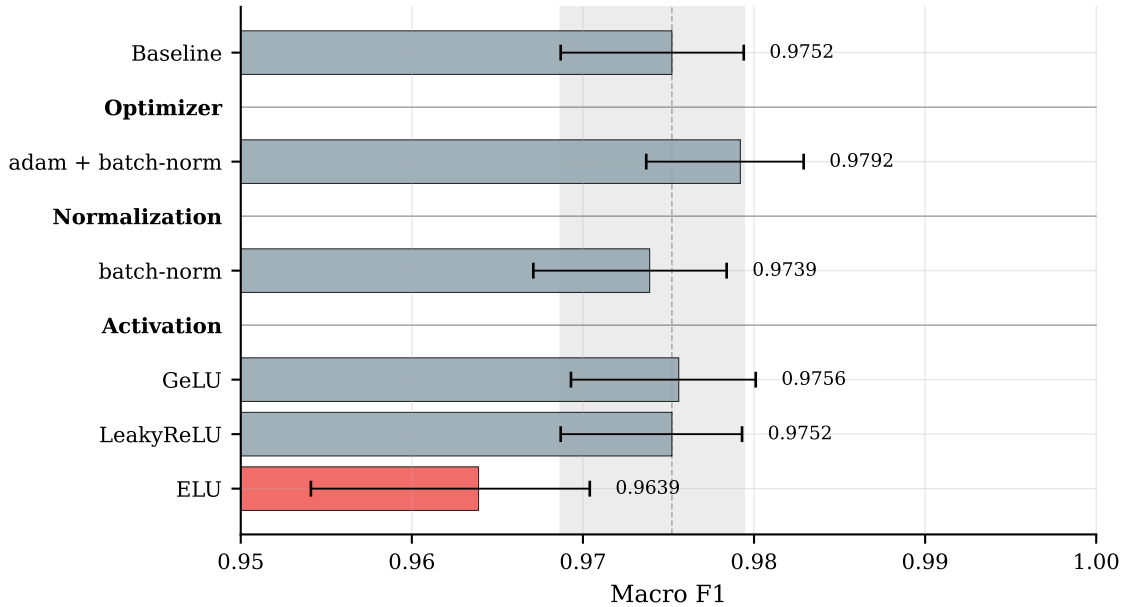


Figure 5.2: Component substitution scan.

The optimizer category pairs Adam [22] with BatchNorm [20]. Bare Adam was tested separately and diverged to a trivial speech-only predictor (not shown). The normalization category swaps WeightNorm for BatchNorm under SGD, which is neutral.

The activation category shows no significant change for GeLU or LeakyReLU. ELU is the only activation with a noticeable difference, and even that is only -0.0113 .

Every substitution variant’s CI overlaps the reference TCN, so no shift in this scan is strict under the CI rule defined in Section 5.1.2. The loss family (focal, class-weighted BCE, MSE, label-smoothed BCE) is similarly flat at ± 0.0003 of the reference and is not plotted.

Hyperparameter sensitivity

The sensitivity scan shifts three numerical parameters (learning rate, dropout, and layer count) along the ranges described in the reference paper [25].

Like the substitution scan, this one produces no strict gains. Tested values and their macro F1 score are shown in Figure 5.3.

Lowering dropout and raising the learning rate both increase macro F1 score by a margin comparable to the Adam+BatchNorm substitution in the previous figure.

The TCN variant with only one layer is the only one in this sensitivity scan whose CI separates from the reference TCN. Its receptive field collapses to 25 frames (≈ 580 ms), well below any other configuration in this thesis. The two- and three-layer variants stay inside sampling noise.

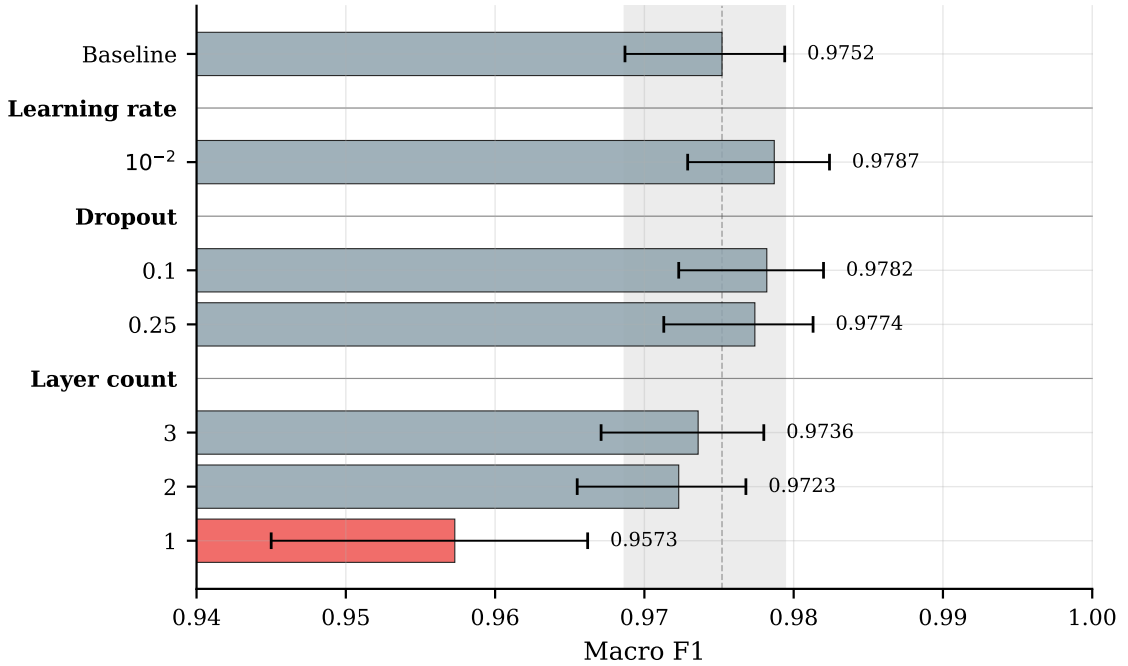


Figure 5.3: Hyperparameter sensitivity scan.

Other numerical hyperparameters were tested but produced flat results across every value (Table 5.2).

Parameter	Tested values	$\Delta F1$
filter count	8, 32	± 0.003
stack count	5	± 0.001
kernel size	3, 7, 9	± 0.003
chunk length	256, 270	± 0.002
batch size	16, 64	± 0.002

Table 5.2: Hyperparameter sensitivity variants whose macro F1 score landed within sampling noise of the reference TCN.

Mel-bin alternatives at 40 and 128 bins do not appear here because they are evaluated in the frontend scan of Section 5.3.2 instead.

Takeaway

One-at-a-time changes within the bounds of the original recipe produce small, scattered effects. Every change landed inside the reference CI or strictly below it. No variant within Lemaire and Holzapfel’s proposed TCN hyperparameter bounds is therefore adopted as a new model.

I have taken the reference TCN as the foundation of the thesis’s proposed method. The following experiments in this section determine its final form.

5.3.2 Spectral frontend

This experiment tries different fixed (non-learned) frontends because different representations of the spectrum may suit the TCN’s first layer better than log-mel alone. Variants cover three families:

1. log-mel bin count: 40, 80, 128
2. log-mel temporal derivatives: Δ and Δ^2
3. energy-normalization alternatives: 20 and 40 MFCCs, PCEN

Δ and Δ^2 are first- and second-order frame-to-frame differences of the log-mel spectrogram [14], concatenated as extra channels alongside the original. Δ doubles the channel count to 160, Δ^2 triples it to 240.

PCEN (per-channel energy normalization) [42] replaces the log compression of the log-mel frontend with a per-channel adaptive automatic gain control followed by a soft power-law compression.

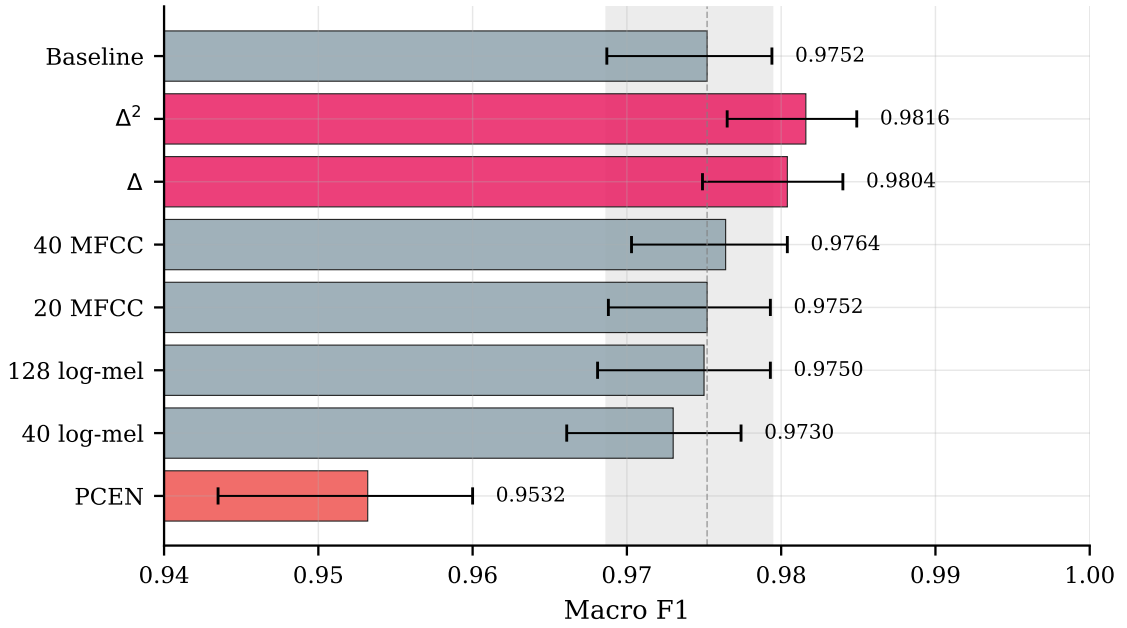


Figure 5.4: Spectral frontend scan.

Results show that frequency resolution is not the discriminative bottleneck. The mel-bin scan is flat, and MFCC confirms the verdict at one-quarter of the input dimensionality.

The temporal-derivative variants are the largest positive moves across the chapter’s scans. The TCN’s first-layer convolution does not extract short-term spectral derivatives as cleanly as a hand-coded delta operator, and pre-computing them gives the backbone cleaner derivative features as input.

PCEN is the only frontend that strictly regresses. Its automatic-gain-control behavior strips level cues that the log-mel z-normalization already preserves, producing a double-normalization that hurts the *Background* class specifically. PCEN replacing the log-mel z-normalization rather than adding to it might recover the lost ground. This was not tested.

The Δ^2 frontend is selected for the combined experiment in Section 5.3.5.

5.3.3 Learned preprocessor

This experiment inserts a learned preprocessor between the log-mel frontend and the TCN body to test whether explicit input-side processing before the backbone helps.

Two variants are tested. A 1-D convolution treats the 80 mel bins as channels and mixes them all at once. A 2-D convolution treats the spectrogram as an image and mixes only neighboring time-frequency cells through a 3×3 kernel. Both variants use a 3-frame causal temporal kernel, so the lookback is identical.

The results are presented in Figure 5.5. The parenthesized number next to each variant’s macro F1 score is a relative parameter count compared to the reference TCN.

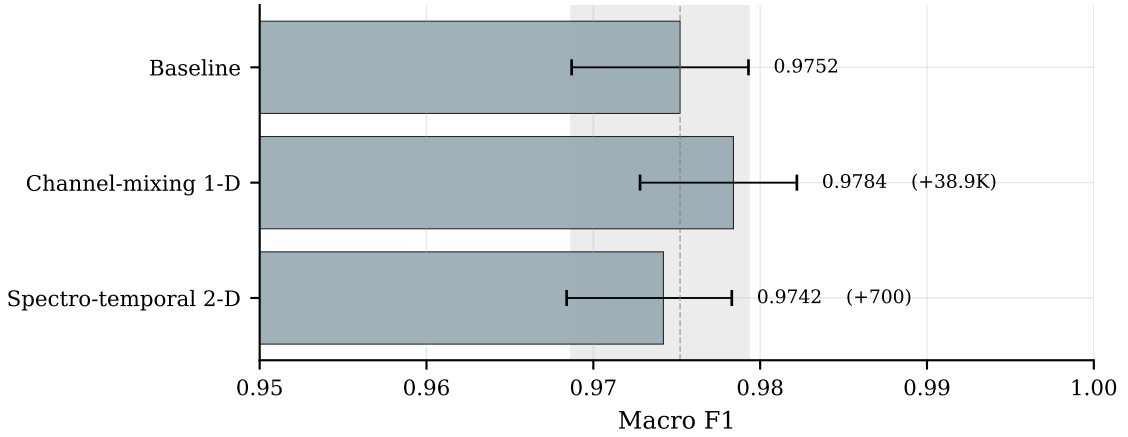


Figure 5.5: Learned-preprocessor scan.

The 1-D variant’s gain is consistent with channel-mixing depth the TCN’s 1×1 input projection cannot provide on its own. The 2-D variant’s dip is consistent with its small spectro-temporal patches duplicating what the backbone’s dilated blocks already construct. Both sit within sampling noise, but the directional gap suggests an architecture-aware preprocessor may help.

The 1-D channel-mixing preprocessor is selected for the combined experiment in Section 5.3.5.

5.3.4 Alternative backbones

This experiment swaps the TCN body for recurrent and attentive alternatives to check whether they fit the task better than the TCN.

Three alternative backbones (GRU, LSTM, Transformer) are tested at two parameter budgets, with the rest of the recipe held constant. The smaller budget targets ~ 30 K parameters, matching the reference TCN. The larger budget targets ~ 130 K parameters, matching the TCN scaled to filter count $F = 32$, included as a same-family reference.

F denotes each backbone’s width parameter. For GRU and LSTM, F is the hidden state size and for the Transformer, F is the model dimension.

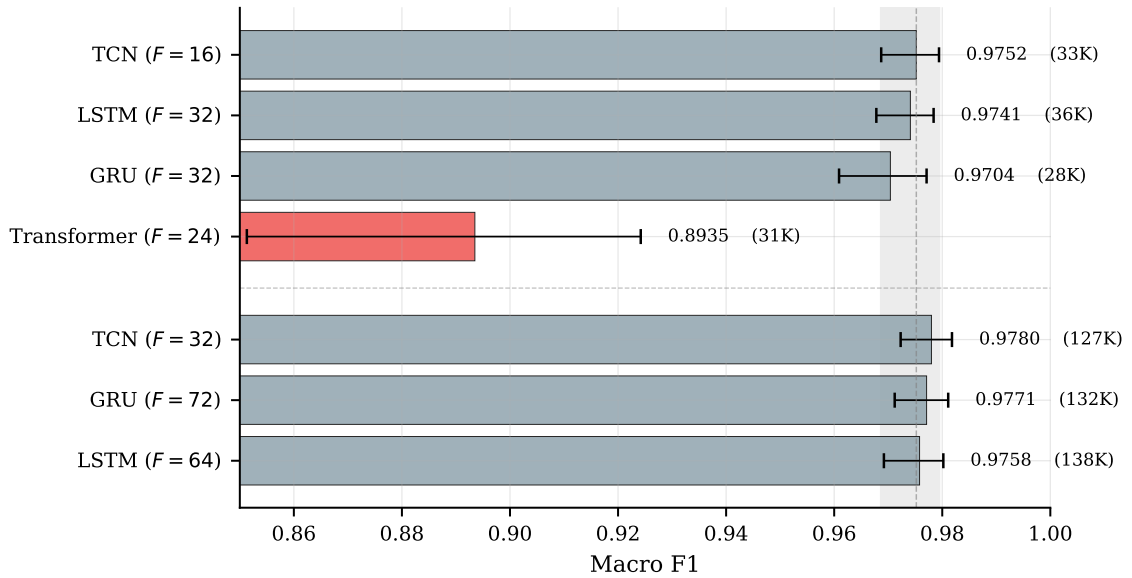


Figure 5.6: Backbone architecture scan.

The TCN family leads at both budgets by score, but every CI in the recurrent cluster overlaps the reference TCN and the others. The lead is direction-stable across the small and large tiers but not strict.

The Transformer is the strict outlier of the experiment: both variants land below the reference CI, and the large variant regresses with scale instead of improving. The large Transformer reaches macro F1 score 0.6942 (omitted from the plot for scale), with the failure concentrated on the *Background* class (recall 0.20, F1 0.32). This work does not provide enough training data for the Transformer, though this hypothesis is not investigated further.

No alternative backbone beats the TCN. The larger TCN at $F = 32$ is carried into the next experiment to test capacity increase together with the frontend and preprocessor winners.

5.3.5 Combined variants

This experiment combines the three best-performing variants from three previous experiments to test whether their individual gains compose. The selected modifications are F (Δ^2 frontend, Section 5.3.2), P (1-D preprocessor, Section 5.3.3), and A (32-filter capacity, Section 5.3.4). The shorthands F, P, and A are used throughout the rest of Section 5.3 to denote these variants.

The pairs (F+P, F+A, P+A) and the triple (F+P+A) are tested. The single-modification variants are not retested here, as their results are already reported.

Figure 5.7 reports the four stacked variants against the reference TCN. The parenthesized number shows each variant’s total parameter count.

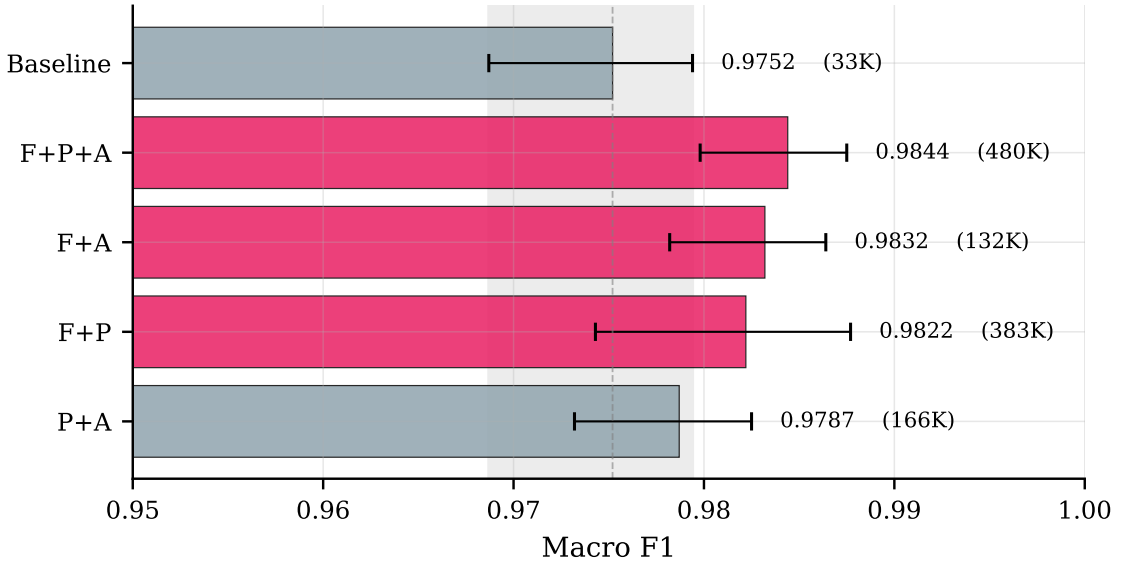


Figure 5.7: Combined modifications.

F+P+A is the only combination whose CI strictly separates from the reference TCN, though F+A and F+P also clear the reference’s upper CI bound by score. It pays roughly fifteen times the reference TCN parameter count (~ 480 K versus ~ 33 K) for a $+0.0092$ macro F1 score gain.

Most of the weight sits in the preprocessor. The 1-D preprocessor’s parameter count scales quadratically with input channel count: on the reference TCN’s 80 log-mel bins it costs ~ 39 K, but on the Δ^2 frontend’s 240 channels it costs ~ 348 K, roughly nine times more. F on its own and A on its own each add at most a few thousand parameters, so P drives the cost of every combination that includes it.

The F+P+A gain recovers 74 % of the sum of the source-experiment standalone gains. Each pair is also sub-additive, most strongly so for P+A (58 % retention), then F+P (73 %), then F+A (87 %). P overlaps with both F and A. F+A is the most independent pair.

Beyond F+P+A, the next experiment turns to the temporal-modeling dimension. It tests stateful and attentive heads on the F+P base, which exhausts the channel-mixing dimensions without scaling the backbone, so the head’s contribution can be measured cleanly.

5.3.6 Temporal head

This experiment adds a stateful or attentive head on top of the F+P TCN base from Section 5.3.5 to test whether more temporal modeling helps beyond what the dilated stack already captures. The “temporal head” in this context represents a small neural network operating on the output logits of the main architecture.

Four candidate heads are tried: a small LSTM ($F = 32$), a small GRU ($F = 32$), a wide GRU ($F = 64$), and causal attention. Figure 5.8 presents the resulting macro F1 scores with added parameter count in parentheses.

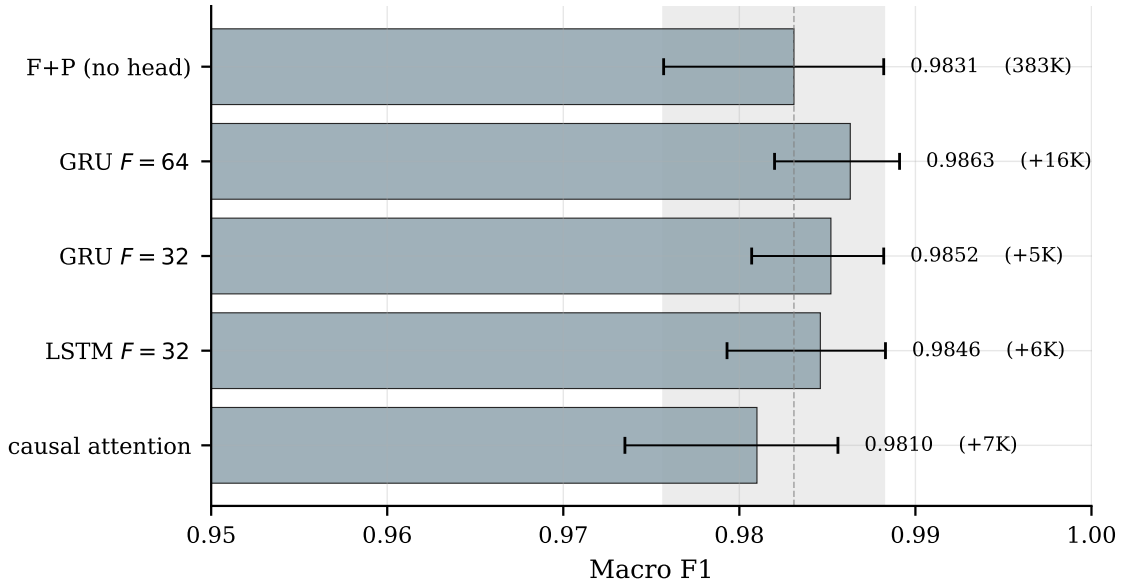


Figure 5.8: Temporal head scan on the F+P TCN base.

No head produces a Δ outside the F+P-base CI. The attention regression echoes the architecture experiment of Section 5.3.4. No tested head adds meaningfully to what F+P already provides.

5.3.7 Conclusion

Both the combined experiment (Section 5.3.5) and the temporal-head experiment (Section 5.3.6) show that further gains over the reference TCN are possible, but come at disproportionate parameter cost. F+P+A wins strictly over the reference TCN by +0.0092 at fifteen times its parameter count. No temporal head on F+P adds anything strict on top of F+P alone.

A representative large variant is still needed for the cross-model comparison in Section 5.5. I pick F+P+LSTM (Δ^2 frontend, 1-D preprocessor, LSTM $F = 32$) and name this variant **TCN-L**.

TCN-L matches F+P+A’s macro F1 score (0.9846 versus 0.9844) at ~ 90 K fewer parameters, and exercises a structurally different dimension than F+P+A: a stateful temporal head on top of the channel-mixing pair, rather than a wider backbone. LSTM is selected as the conventional streaming head. Among the temporal heads the choice is noise-level on macro F1 score.

TCN-L is analyzed alongside the reference classifiers from Section 5.5 onward.

Section 5.4 reverses the direction and asks how much computational cost can be shed at comparable performance. Section 5.5 brings all of these experimental variants back under stress conditions and on the cost dimensions, where the differences sharpen.

5.4 Proposed lightweight variant

The previous section established that further gains in effectiveness over the reference TCN cost a disproportionate parameter count. I therefore propose a lightweight classifier matching the reference TCN’s macro F1 score at a fraction of its parameter count and computational cost.

This section presents the path from the reference TCN to the proposed variant. Rather than exploring the full cost-accuracy space, I follow a single greedy path: at each step the cheapest configuration whose macro F1 score remains within CI of the previous step is retained.

Figure 5.9 illustrates the whole experimental path in the (filter count F , stack count S , macro F1) space, with each step described below.

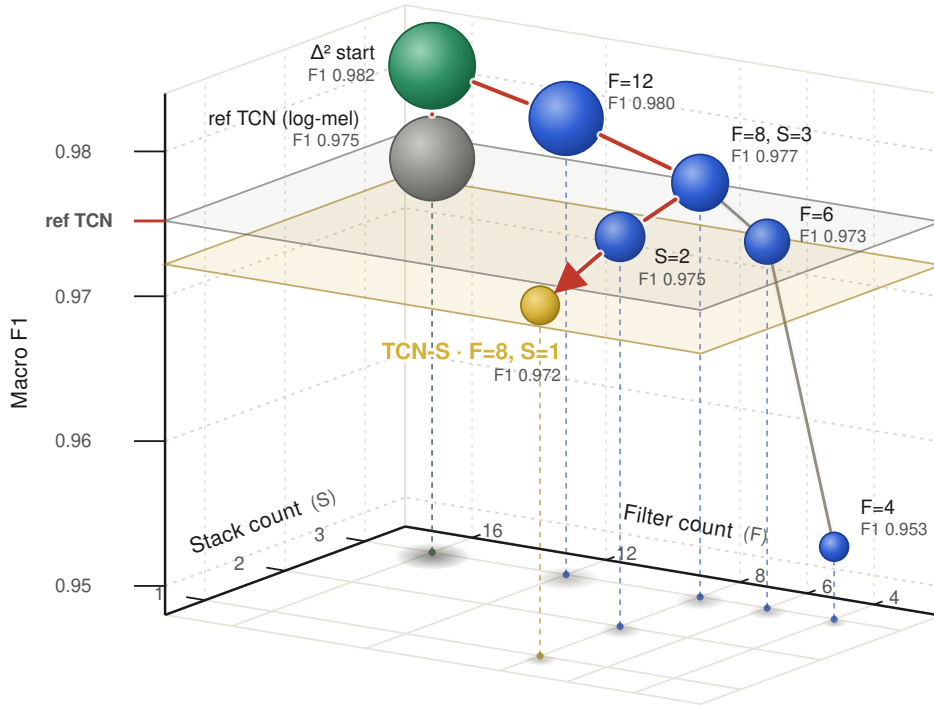


Figure 5.9: Experimental path from reference TCN to TCN-S.

Step 0: starting point choice. The frontend scan of Section 5.3.2 identified Δ^2 as the strongest available change, at near-equal parameter count. Of the eight frontends tested there, only the temporal-derivative family (Δ and Δ^2) moved the score beyond sampling noise. I therefore chose Δ^2 as the starting point for the cost reduction that follows.

Step 1: filter count. The filter count scan was performed on the Δ^2 starting variant. Of the four filter counts tested ($F \in \{4, 6, 8, 12\}$), only $F = 4$ dropped notably. $F = 8$ is picked because halving the number of filters left the macro F1 score almost the same as the reference TCN and reduced its parameter count to almost 1/3. With $F = 8$ fixed, the stack count is next.

Step 2: stack count. The stack count scan was performed on the $F = 8$ variant. Of the three stack counts tested ($S \in \{1, 2, 3\}$), all three scored almost the same. $S = 1$ is picked because shrinking the stack count to 1 left the macro F1 score just below the reference TCN’s and halved the parameter count again.

The resulting single-stack configuration, named **TCN-S**, has ~ 4.6 K trainable parameters, roughly 1/7 the reference TCN’s count. Its macro F1 score of 0.9722 on the test split sits 0.0030 below the reference TCN, inside the reference CI.

5.5 Analyses

With three TCN variants in hand (reference, TCN-L, TCN-S) and three traditional models (decision tree, GMM, SVM), this section runs the analyses that separate them. The test split puts the TCN family within roughly 0.01 macro F1 score of each other and the traditional models ten points below. Meaningful within-family differences live on the cost dimensions and on behavior under stress. Section 5.5.1 reports the measured wall-clock cost from a single-threaded CPU run on the deployment target. Section 5.5.2 complements it with hardware-independent complexity metrics. Section 5.5.3 walks through a manual behavioral review on the critical subset.

5.5.1 CPU benchmark

This subsection benchmarks the wall-clock cost of every classifier. A single run per model records per-decision time and memory footprint. The three analyses that follow read different angles off the same data: per-decision cost, latency decomposition, and the cost vs effectiveness trade-off.

Methodology

Measurements run on a single CPU thread of an Intel i5-8300H (2.30 GHz). The scikit-learn classifiers are single-threaded by default, so PyTorch’s intra-op thread count is pinned to one to match. All neural variants run through the streaming interface of Section 3.5, so the cost numbers reflect on-line deployment.

Each model is fed seeded Gaussian noise at its native hop length: 10 ms for the decision tree, 15 ms for GMM and SVM, and 23.2 ms for the TCN family. Synthetic input decouples the timing from disk I/O and audio decoding, and every model sees the same input.

Each run consists of 10 warm-up decisions (discarded), then 5 trials of 200 timed decisions each. The median is taken over all 1 000 timed decisions.

Δ RSS is the memory footprint added by the loaded model: the peak resident-set size during the run minus a pre-load baseline.

Per-decision cost

Table 5.3 reports three per-decision metrics:

- **Computational delay:** median wall-clock time per decision once the input audio is buffered, the implementation counterpart to the algorithmic delay of Section 2.3.
- **RTF:** audio seconds processed per wall-clock second.

- **ΔRSS** : memory footprint added by the loaded model (defined above).

Model	comp. delay (ms)	RTF	Δrss (MB)
GMM	3.94	3.8×	3
DT	4.15	2.4×	368
TCN-S	4.38	5.3×	13
TCN	7.18	3.2×	8
SVM	7.66	2.0×	9
TCN-L	11.07	2.1×	23

Table 5.3: Single-thread CPU benchmark on i5-8300H.

These metrics feed the two analyses below. TCN-S has the lowest computational delay of any neural variant, satisfying the wall-clock side of the cost-reduction goal from Section 5.4.

Latency decomposition

User-observable latency has two components: algorithmic delay (**AD**, defined in Section 2.3) and computational delay (**CD**, the median from Table 5.3). Table 5.4 sums them under two conditions: steady-state total latency (**TL**, AD + CD per decision) and time to first output (**TFO**, the cold-start sum that uses the full analysis window instead of one hop).

Model	AD (ms)	CD (ms)	TL (ms)	TFO (ms)
DT	10.0	4.15	14.2	24.2
GMM	15.0	3.94	18.9	33.9
SVM	15.0	7.66	22.7	37.7
TCN-S	23.2	4.38	27.6	50.8
TCN	23.2	7.18	30.4	53.6
TCN-L	23.2	11.07	34.3	57.5

Table 5.4: Per-decision latency and time to first output.

For every model, algorithmic delay is the larger component, with computational delay adding at most 11 ms on top (TCN-L). Time to first output is at most 57.5 ms (TCN-L), incurred only once at stream start.

Cost vs effectiveness

The metrics above describe cost alone. Pairing them with the test-split macro F1 score of each model, as in Figure 5.10, reveals the cost-vs-effectiveness trade-off space.

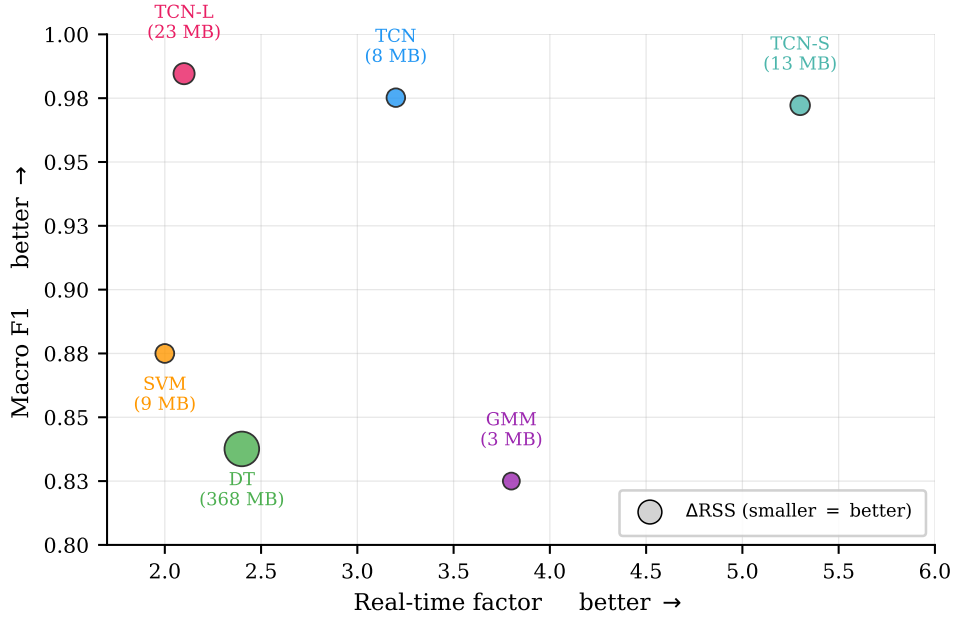


Figure 5.10: Cost vs effectiveness trade-off.

Every model clears the $2\times$ real-time floor at one CPU thread, so the choice between them is on cost rather than streaming feasibility. TCN-S is the real-time-factor winner, about 40% above the next model and 65% above the reference TCN. GMM holds the memory corner, two orders of magnitude less than the decision tree and several times less than any neural model.

5.5.2 Symbolic complexity

The measured numbers from the previous section are hardware-specific. This subsection complements them with hardware-independent complexity metrics derived from each model’s architecture.

Reported metrics

The values come from three sources: the trained models, the streaming inference code, and formulas over each model’s architecture.

Parameter counts are read directly from the trained models. For the TCN family, PyTorch’s parameter iterator gives the count. For the traditional models, the trained scikit-learn estimators expose the relevant sizes: tree node count for the decision tree, support-vector matrix size for SVM, and mixture parameters for GMM.

The number of multiply-accumulate operations (MACs) per output frame follows from how each model is actually run in streaming mode. The TCN family’s backbone is stateless and re-runs over its full receptive-field buffer on every decision, so MACs per frame equal the off-line per-frame cost times the receptive-field length in feature frames. For the traditional models, MACs per frame count the operations on the active decision path: one comparison per tree-depth level for the decision tree, the kernel evaluation across all support vectors

for SVM, or the log-likelihood computation across all mixture components for GMM. The rolling-decision smoothing adds a small constant on top.

The receptive field for the TCN family follows Equation 3.7 with the per-variant kernel size, blocks per stack, and number of stacks substituted in. For the traditional models, the receptive field is the feature-extractor window (300 ms for the decision tree, 1 s for GMM and SVM).

Table 5.5 reports the three resulting columns:

- **Parameter count:** total number of trainable weights.
- **MACs/frame:** multiply-accumulate operations executed to produce one output frame, including the receptive-field re-run that the current streaming implementation performs per decision.
- **Receptive field (RF):** temporal span of input the model conditions on per decision.

Model	params	MACs/frame	RF (ms)
DT	4.34 M	178	300
GMM	474	216	1000
SVM	521 K	469 K	1000
TCN-S	4.6 K	545 K	2810
TCN	33 K	11.6 M	8382
TCN-L	389 K	137 M	8382

Table 5.5: Symbolic complexity.

Streaming MAC count varies by orders of magnitude across the family. The decision tree and GMM are the cheapest, three to six orders of magnitude below the rest. The decision tree’s count is mostly tree-branch comparisons rather than dense MACs, so the wall-clock benchmark of Table 5.3 is a more reliable cost ordering at the low end.

Within the TCN family, TCN-S is roughly $21\times$ below the reference TCN in MACs. The wall-clock reduction is smaller ($1.6\times$ faster), since pipeline overhead does not scale with MAC count. TCN-L is the most expensive variant, with most of the cost in the LSTM head’s persistent-state matrix multiplication.

Receptive fields span an order of magnitude across the family, with TCN-S at roughly one-third of the reference TCN’s window. TCN-S combines that shorter receptive field with halved filter count to reach the $21\times$ MAC reduction at only 0.003 macro F1 score loss, so the reduced capacity suffices for most of the test distribution.

Parameter vs compute

Figure 5.11 plots parameter count against streaming MACs per output frame on log-log axes to visualize how the two cost dimensions decouple across the family.

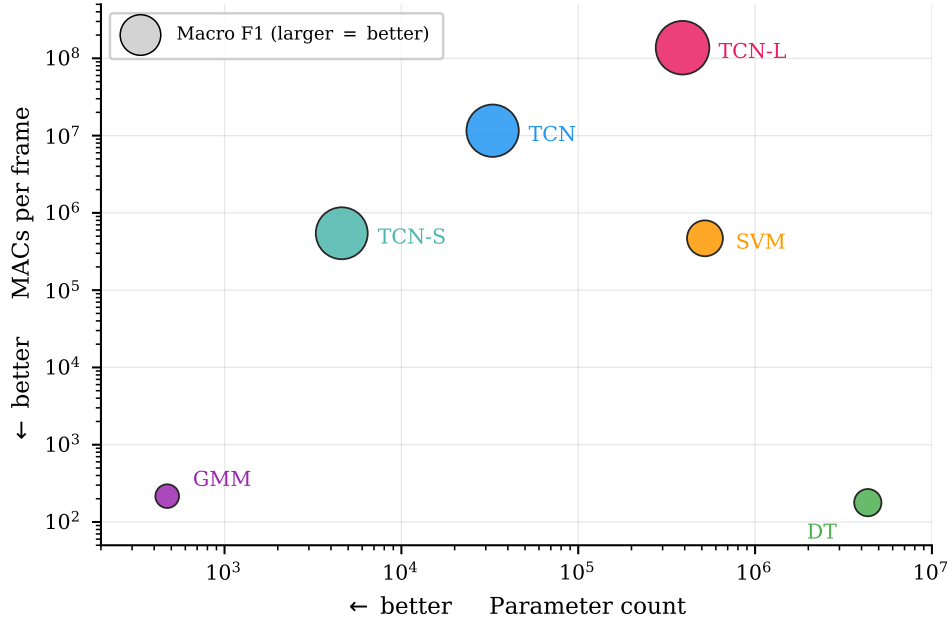


Figure 5.11: Parameter count against streaming MACs (log-log).

The decision tree sits in the high-parameter, low-compute corner (large tree, comparison-only inference). The TCN family occupies the low-parameter, high-compute corner (small weight tensors with a receptive-field re-run on every decision). GMM and SVM bridge the two diagonals.

5.5.3 Critical subset check

The critical subset (Section 4.5) was built towards the end of work on this thesis to sanity-check the models through manual review rather than through aggregate numbers alone. Predictions are plotted over a hand-labeled ground truth. The sanity-check pair, the misclassified-background clip, and the extreme-music clip show all six classifiers. The remaining figures show TCN-S alone, the proposed method of this thesis. Each figure below shows 15s of one representative clip. The rest of the subset followed the same trajectory: every remaining clip either worked as expected, or failed with the same *Background-mistaken-for-Music* pattern shown in Figure 5.14. This subsection summarizes the findings.

Figure 5.12 shows every model classifying a 15 s pop-music clip and a clean-speech recording.

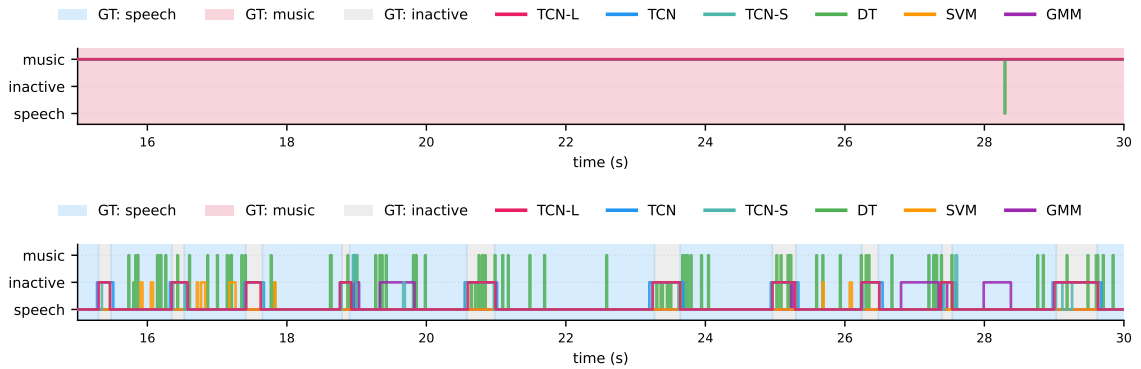


Figure 5.12: Classifier decisions on a pop music clip (top) and a clean speech recording (bottom).

Every model classifies both clips almost perfectly. The decision tree misclassifies one frame on the pop clip, and the SVM scatters a few frames on speech. The decision tree’s output also flickers more than the others, traced to its 10 ms frame length (further discussed in Section 6.3).

The next two figures each pair a *Background* clip with the same clip mixed with *Speech*.

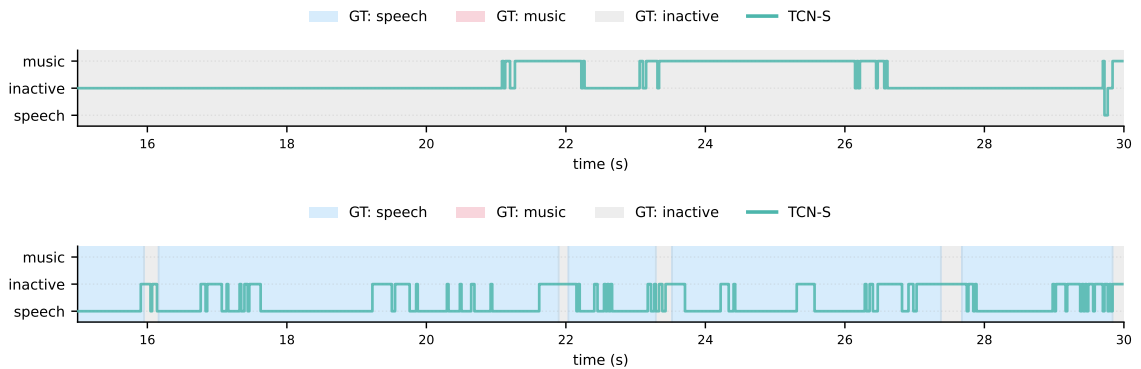


Figure 5.13: TCN-S on a correctly classified *Background* clip (top) and the same clip with *Speech* overlaid (bottom).

Figure 5.13 is the expected case and slightly exceeds expectations. When TCN-S labels the noise as *Background* on its own, it also recognizes *Speech* mixed over the same noise at -3 dB SNR.

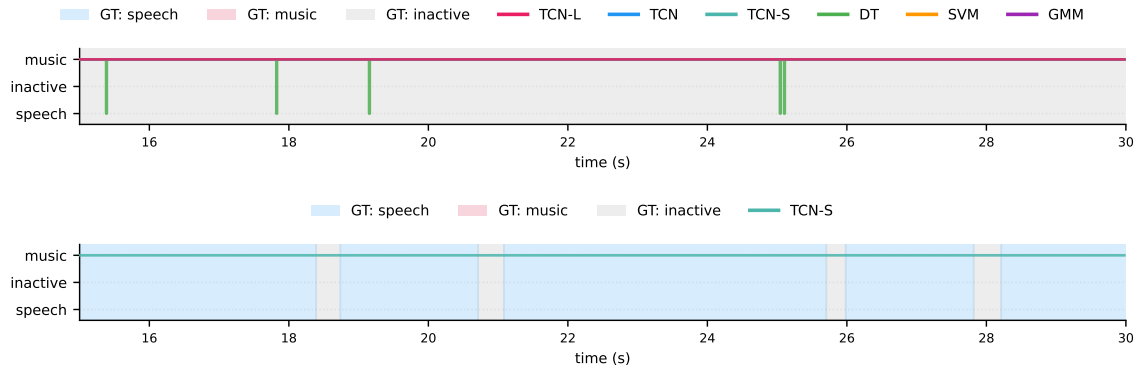


Figure 5.14: Misclassified *Background* clip across all six classifiers (top) and the same clip with *Speech* overlaid on TCN-S (bottom).

Figure 5.14 is the opposite case. Every classifier labels the unknown background as *Music*, and TCN-S sticks to *Music* even when speech enters the mix. This is the largest negative finding of the review: the dataset’s *Background* class under-represents the variety of ambient sound a real deployment encounters. The remedy is discussed in Section 6.3.

Figure 5.15 shows classifier decisions on a 400 BPM electronic clip.

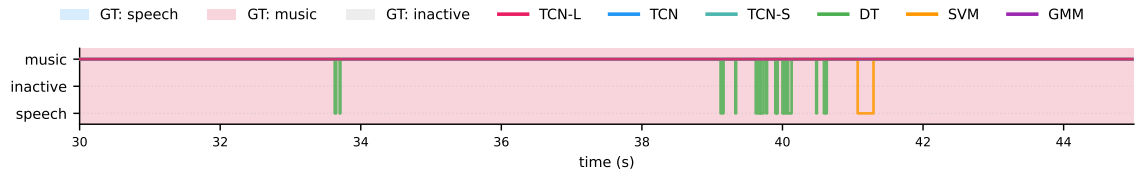


Figure 5.15: Classification decisions on extreme music.

Manual review of this and other extreme-music recordings (Belgian jumpup, beatbox) was almost perfect, a positive surprise.

Figure 5.16 groups speech over music, multi-speaker recordings, and multi-speaker over music.

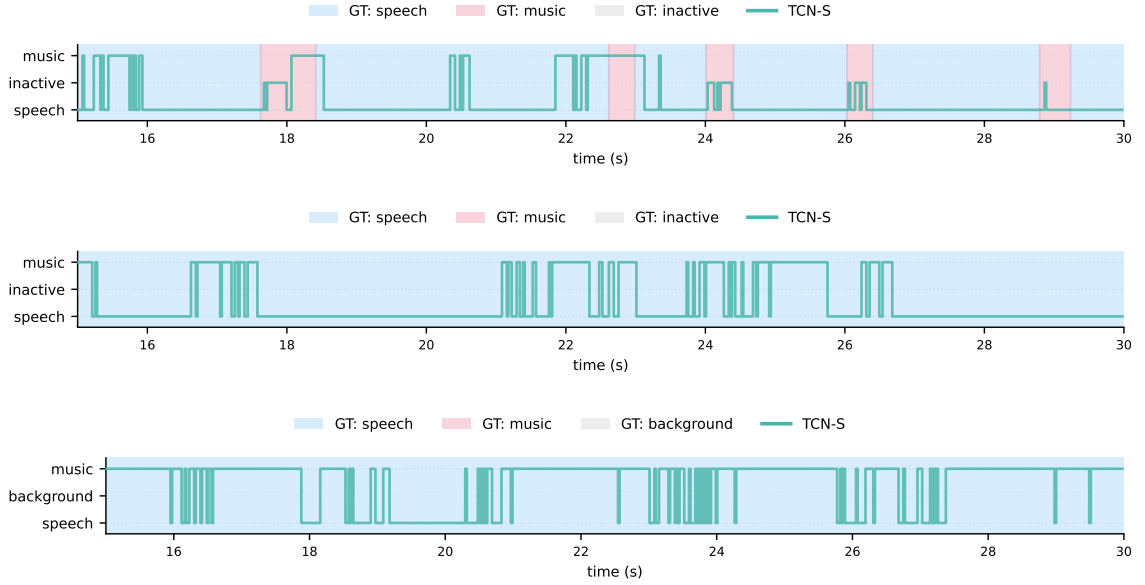


Figure 5.16: TCN-S decisions on speech over music (top), multi-speaker recording (middle), and multi-speaker over music (bottom).

Figure 5.16 reveals a second coverage limitation. TCN-S handles speech over music (top) with brief flicker at transitions. Multi-speaker speech alone (middle) already strains it. Adding music on top of multi-speaker speech (bottom) overwhelms the model entirely.

The walk-through served its sanity-check purpose: every model classifies the content it was capable of handling as expected. It also uncovered a dataset gap: when a classifier cannot handle an ambient sound, it labels the clip as *Music*, and TCN-S keeps that label even when speech enters the mix. Expanding the dataset’s *Background* coverage is taken up as future work in Section 6.4.

Chapter 6

Conclusion

This chapter summarizes the work, its contributions, its limitations, and directions for follow-up work.

6.1 Summary

The primary goal of this thesis was to design and implement a real-time speech and music classifier. The real-time requirement invited strict causality as the self-imposed constraint. The reference methods address this task as 2-class speech-vs-music, and this thesis extends them to a 3-class formulation (*Speech*, *Music*, *Background*) by adding silence and ambient noise as a third class.

A deterministic dataset construction pipeline was developed and produced a ~ 100 -hour training dataset from four public sources, paired with a manually labeled critical subset for behavioral review. Four reference classifiers from the literature were implemented: a decision tree from Lavner and Ruinskiy, a Gaussian mixture model and a support vector machine from Khonglah and Prasanna, and a causal Temporal Convolutional Network (TCN) from Lemaire and Holzapfel. The causal TCN reached macro F1 score 0.9752, ten percentage points above the strongest traditional reference.

Two experimental phases followed. The effectiveness phase explored the TCN’s design dimensions and produced TCN-L (~ 389 K parameters, macro F1 score 0.9846) as the representative high-capacity variant at roughly $12\times$ the reference parameter count.

After it became clear that further accuracy gains required much larger models, the cost phase reversed direction and asked how much computational cost could be shed at comparable accuracy. That phase produced **TCN-S** (~ 4.6 K parameters, macro F1 score 0.9722), which matches the reference TCN within 0.003 macro F1 score at $1/7$ the parameter count and roughly $21\times$ less per-frame compute. A single-thread CPU benchmark confirmed that every implemented classifier runs above $2\times$ real-time, with TCN-S leading the family at $5.3\times$.

6.2 Contributions

The primary contribution of this thesis is the proposed **TCN-S**, a causal classifier (~ 4.6 K parameters) that matches the reference TCN within 0.003 macro F1 score at $1/7$ the parameter count and roughly $21\times$ less per-frame compute. A second contribution is the **dataset construction pipeline**: a deterministic build that produced the ~ 100 -hour

training dataset from four public sources, designed so that adding new sources or download adapters extends the dataset without redesigning the pipeline.

The remaining contributions support the two above:

- **Implementations of three traditional references** extended from 2-class to 3-class streaming: a decision tree from Lavner and Ruinskiy, and a Gaussian mixture model and a support vector machine from Khonglah and Prasanna.
- **Causal TCN implementation** following Lemaire and Holzapfel, plus the derived TCN-L (~ 389 K parameters, macro F1 score 0.9846) as a high-accuracy comparison point.
- **Systematic experimental program** over the TCN’s design space: sensitivity ablation, spectral frontend, learned preprocessor, alternative backbones, combined variants, temporal heads, and the cost-reduction sweep that derives TCN-S.
- **Cross-model analyses:** a single-thread CPU benchmark, symbolic complexity metrics (parameters, streaming MACs, receptive field), and a critical-subset walk-through for manual behavioral review.

The poster and presentation video required by the assignment are submitted alongside the thesis, together with the source code needed to reproduce or build upon it.

6.3 Limitations

Two main limitations of this work are noted below.

Per-model frame-length differences. Each implemented model emits decisions at its own cadence: 10 ms for the decision tree, 15 ms for GMM and SVM, and 23.2 ms for the TCN family. These hops follow each model’s on-line adaptation rather than the reference papers verbatim, and they make the decision tree’s predictions look more jittery in side-by-side timeline plots than the others, even when accuracy is comparable. A uniform-frame-rate re-evaluation (Section 6.4) would remove this artifact.

Dataset coverage gap on *Background*. The critical-subset review (Section 5.5.3) surfaced a gap in the assembled dataset: ambient sounds the classifier cannot recognize collapse onto *Music*, and TCN-S keeps that label even when speech enters the mix. The gap traces to the limited *Background* variety in the public sources used and is the main motivation for dataset expansion described in Section 6.4.

6.4 Follow-up work

The work leaves several directions open for follow-up. Three are outlined below.

Dataset expansion. The assembled dataset is large enough to support training but does not yet cover the full variety of inputs the classifier will encounter, with *Background* as the most prominent gap. Expanding training coverage across all three classes may allow a re-trained classifier to close most of the visible robustness gap. A separate evaluation dataset drawn from sources disjoint from the training set would in turn quantify generalization beyond the training distribution.

Codec integration and perceptual-quality testing. The natural next step is integration into a speech or audio codec, e.g., the speech/music classifier of EVS [10], where

TCN-S sits well within the codec’s per-frame CPU budget. A codec variant that swaps in a classifier from this thesis can then be compared against the production setup by objective and subjective perceptual-quality testing. This comparison answers the question macro F1 score cannot: whether the proposed classifier yields a better-sounding codec, not just a better-scoring classifier.

Uniform frame-rate evaluation. The implemented classifiers emit decisions at different cadences (Section 6.3). Re-evaluating every model at one shared frame rate would normalize the visual cadence in side-by-side timeline plots and the per-frame metric weighting, removing the jitter artifact.

Bibliography

- [1] AL SHOSHAN, A. I. Speech and Music Classification and Separation: A Review. *Journal of King Saud University - Engineering Sciences*, 2006, vol. 19, no. 1, p. 95–133. ISSN 1018-3639. Available at: <https://www.sciencedirect.com/science/article/pii/S101836391830850X>.
- [2] BAI, S.; KOLTER, J. Z. and KOLTUN, V. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. 2018. Available at: <https://arxiv.org/abs/1803.01271>. [cit. 2026-04-19]. ArXiv preprint arXiv:1803.01271 [cs.LG].
- [3] BENZEGHIBA, M.; DE MORI, R.; DEROO, O.; DUPONT, S.; ERBES, T. et al. Automatic speech recognition and speech variability: A review. *Speech Communication*, 2007, vol. 49, 10-11, p. 763–786. ISSN 0167-6393. Available at: <https://www.sciencedirect.com/science/article/pii/S0167639307000404>. Intrinsic Speech Variations.
- [4] BHATTACHARJEE, M.; PRASANNA, S. R. M. and GUHA, P. *Time-Frequency Audio Features for Speech-Music Classification*. 2018. Available at: <https://arxiv.org/abs/1811.01222>. [cit. 2026-05-09]. ArXiv preprint arXiv:1811.01222 [eess.AS].
- [5] BISHOP, C. M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. Information Science and Statistics. ISBN 978-0-387-31073-2.
- [6] BREDIN, H. Pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe. In: *Proc. INTERSPEECH 2023*. Dublin, Ireland: ISCA, 2023, p. 1983–1987. Available at: https://www.isca-archive.org/interspeech_2023/bredin23_interspeech.html.
- [7] CHOI, K.; FAZEKAS, G.; SANDLER, M. and CHO, K. Convolutional Recurrent Neural Networks for Music Classification. In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. New Orleans, LA, USA: IEEE, 2017, p. 2392–2396.
- [8] DAVISON, A. C. and HINKLEY, D. V. *Bootstrap Methods and their Application*. Cambridge: Cambridge University Press, 1997. Cambridge Series in Statistical and Probabilistic Mathematics. ISBN 978-0-521-57471-6.
- [9] DEFFERRARD, M.; BENZI, K.; VANDERGHEYNST, P. and BRESSON, X. FMA: A Dataset for Music Analysis. In: CUNNINGHAM, S. J.; DUAN, Z.; HU, X. and TURNBULL, D., ed. *18th International Society for Music Information Retrieval Conference*. Suzhou, China: ISMIR, 2017, p. 316–323.

- [10] DIETZ, M.; MULTRUS, M.; EKSLER, V.; MALENOVSKÝ, V.; NORVELL, E. et al. Overview of the EVS codec architecture. In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. South Brisbane, QLD, Australia: IEEE, 2015, p. 5698–5702.
- [11] EFRON, B. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*. Institute of Mathematical Statistics, 1979, vol. 7, no. 1, p. 1–26.
- [12] ELMAN, J. L. Finding Structure in Time. *Cognitive Science*, 1990, vol. 14, no. 2, p. 179–211.
- [13] EYBEN, F. *Real-time Speech and Music Classification by Large Audio Feature Space Extraction*. Cham, Switzerland: Springer International Publishing, 2016. Springer Theses. ISBN 978-3-319-27298-6. Available at: <http://dx.doi.org/10.1007/978-3-319-27299-3>.
- [14] FURUI, S. Speaker-independent isolated word recognition using dynamic features of speech spectrum. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1986, vol. 34, no. 1, p. 52–59.
- [15] GUYON, I. and ELISSEEFF, A. An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 2003, vol. 3, p. 1157–1182. ISSN 1532-4435.
- [16] HAN, J.; KAMBER, M. and PEI, J. Data Preprocessing. In: *Data Mining: Concepts and Techniques (Third Edition)*. Third Editionth ed. Boston: Morgan Kaufmann, 2012, chap. 3, p. 83–124. The Morgan Kaufmann Series in Data Management Systems. ISBN 978-0-12-381479-1. Available at: <https://www.sciencedirect.com/science/article/pii/B9780123814791000034>.
- [17] HARRIS, C. R.; MILLMAN, K. J.; WALT, S. J. van der; GOMMERS, R.; VIRTANEN, P. et al. Array programming with NumPy. *Nature*, 2020, vol. 585, no. 7825, p. 357–362. ISSN 1476-4687.
- [18] HIRVONEN, T. Speech/Music Classification of Short Audio Segments. In: *2014 IEEE International Symposium on Multimedia*. Taichung, Taiwan: IEEE Computer Society, 2014, p. 135–138. ISBN 978-1-4799-4311-1. Available at: <https://ieeexplore.ieee.org/document/7033009/>.
- [19] HOCHREITER, S. and SCHMIDHUBER, J. Long Short-Term Memory. *Neural Computation*, november 1997, vol. 9, no. 8, p. 1735–1780. ISSN 0899-7667. Available at: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [20] IOFFE, S. and SZEGEDY, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Lille, France: PMLR, 2015, vol. 37, p. 448–456. Proceedings of Machine Learning Research.
- [21] KHONGLAH, B. K. and PRASANNA, S. R. M. Speech / music classification using speech-specific features. *Digital Signal Processing*, 2016, vol. 48, p. 71–83. ISSN 1051-2004. Available at: <https://www.sciencedirect.com/science/article/pii/S1051200415002730>.

- [22] KINGMA, D. P. and BA, J. Adam: A Method for Stochastic Optimization. In: *3rd International Conference on Learning Representations (ICLR)*. San Diego, CA, USA: [b.n.], 2015. Available at: <https://arxiv.org/abs/1412.6980>.
- [23] LAVNER, Y. and RUINSKIY, D. A decision-tree-based algorithm for speech/music classification and segmentation. *EURASIP Journal on Audio, Speech, and Music Processing*. Springer Nature, 2009, vol. 2009, no. 239892, p. 1–14. ISSN 1687-4722.
- [24] LECUN, Y.; BOTTOU, L.; BENGIO, Y. and HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998, vol. 86, no. 11, p. 2278–2324.
- [25] LEMAIRE, Q. and HOLZAPFEL, A. Temporal Convolutional Networks for Speech and Music Detection in Radio Broadcast. In: FLEXER, A.; PEETERS, G.; URBANO, J. and VOLK, A., ed. *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. Delft, The Netherlands: ISMIR, 2019, p. 229–236. Available at: <https://archives.ismir.net/ismir2019/paper/000026.pdf>.
- [26] MAKHOUL, J. Linear prediction: A tutorial review. *Proceedings of the IEEE*, 1975, vol. 63, no. 4, p. 561–580.
- [27] MCFEE, B.; RAFFEL, C.; LIANG, D.; ELLIS, D. P. W.; MCVICAR, M. et al. Librosa: Audio and Music Signal Analysis in Python. In: HUFF, K. and BERGSTRA, J., ed. *Proceedings of the 14th Python in Science Conference (SciPy)*. Austin, TX, USA: SciPy, 2015, p. 18–24.
- [28] NEUENDORF, M.; MULTRUS, M.; RETTELACH, N.; FUCHS, G.; ROBILLIARD, J. et al. The ISO/MPEG Unified Speech and Audio Coding Standard—Consistent High Quality for All Content Types and at All Bit Rates. *Journal of the Audio Engineering Society*, 2013, vol. 61, no. 12, p. 956–977. ISSN 1549-4950. Available at: <https://www.aes.org/e-lib/browse.cfm?elib=17074>.
- [29] PANAYOTOV, V.; CHEN, G.; POVEY, D. and KHUDANPUR, S. Librispeech: an ASR corpus based on public domain audio books. In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. South Brisbane, QLD, Australia: IEEE, 2015, p. 5206–5210.
- [30] PASZKE, A.; GROSS, S.; MASSA, F.; LERER, A.; BRADBURY, J. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: WALLACH, H.; LAROCHELLE, H.; BEYGELZIMER, A.; BUC, F. d’Alché; FOX, E. et al., ed. *Advances in Neural Information Processing Systems 32 (NeurIPS)*. Red Hook, NY, USA: Curran Associates, Inc., 2019, p. 8024–8035.
- [31] PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A.; MICHEL, V.; THIRION, B. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011, vol. 12, no. 85, p. 2825–2830. ISSN 1532-4435. Available at: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [32] PLAQUET, A. and BREDIN, H. Powerset multi-class cross entropy loss for neural speaker diarization. In: *Proc. INTERSPEECH 2023*. Dublin, Ireland: ISCA, 2023, p. 3222–3226. Available at: https://www.isca-archive.org/interspeech_2023/plaquet23_interspeech.html.

- [33] PUDIL, P.; NOVOTNÁ, J. and KITTLER, J. Floating search methods in feature selection. *Pattern Recognition Letters*, 1994, vol. 15, no. 11, p. 1119–1125. ISSN 0167-8655. Available at: <https://www.sciencedirect.com/science/article/pii/0167865594901279>.
- [34] RÉMY, P. *Keras-tcn: Keras Temporal Convolutional Network* GitHub repository. 2018. Available at: <https://github.com/philipperemy/keras-tcn>. [cit. 2026-05-07].
- [35] RUMELHART, D. E.; HINTON, G. E. and WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, 1986, vol. 323, no. 6088, p. 533–536.
- [36] SAUNDERS, J. Real-time discrimination of broadcast speech/music. In: *1996 IEEE International Conference on Acoustics, Speech, and Signal Processing Conference Proceedings*. Atlanta, GA, USA: IEEE, 1996, vol. 2, p. 993–996.
- [37] SCHEIRER, E. and SLANEY, M. Construction and evaluation of a robust multifeature speech/music discriminator. In: *1997 IEEE International Conference on Acoustics, Speech, and Signal Processing*. Munich, Germany: IEEE, 1997, vol. 2, p. 1331–1334.
- [38] SILERO TEAM. *Silero VAD: Pre-Trained Enterprise-Grade Voice Activity Detector* GitHub repository. 2024. Available at: <https://github.com/snakers4/silero-vad>. [cit. 2026-04-21].
- [39] THIEMANN, J.; ITO, N. and VINCENT, E. The Diverse Environments Multi-channel Acoustic Noise Database (DEMAND): A database of multichannel environmental noise recordings. *Proceedings of Meetings on Acoustics*, june 2013, vol. 19, no. 1, p. 035081. ISSN 1939-800X.
- [40] TZANETAKIS, G. and COOK, P. Musical Genre Classification of Audio Signals. *IEEE Transactions on Speech and Audio Processing*. IEEE, 2002, vol. 10, no. 5, p. 293–302.
- [41] VIRTANEN, P.; GOMMERS, R.; OLIPHANT, T. E.; HABERLAND, M.; REDDY, T. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 2020, vol. 17, no. 3, p. 261–272. ISSN 1548-7091.
- [42] WANG, Y.; GETREUER, P.; HUGHES, T.; LYON, R. F. and SAUROUS, R. A. Trainable Frontend for Robust and Far-Field Keyword Spotting. In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. New Orleans, LA, USA: IEEE, 2017, p. 5670–5674.
- [43] ZHOU, M.; XU, S.; LIU, Z.; WANG, Z.; YU, F. et al. CCMusic: An Open and Diverse Database for Chinese Music Information Retrieval Research. *Transactions of the International Society for Music Information Retrieval*. Ubiquity Press, Mar 2025, vol. 8, no. 1, p. 22–38. ISSN 2514-3298. Available at: <https://doi.org/10.5334/tismir.194>.

Appendix A

Dataset sources

This appendix enumerates the HuggingFace datasets from which the mid and full subsets of Chapter 4 are built.

- LibriSpeech ASR [29], clean and dirty speech: https://hf.co/datasets/openslr/librispeech_asr
- FMA [9], six music genres (instrumental, electronic, pop, rock, hip-hop, folk): <https://hf.co/datasets/rpmon/fma-genre-classification>
- Bel Canto [43], a cappella music: https://hf.co/datasets/ccmusic-database/bel_canto
- DEMAND [39], noise (background): <https://hf.co/datasets/voice-biomarkers/DEMAND-acoustic-noise>

The four synthetic speech subclasses (multi-speaker speech, speech over music, multi-speaker speech over music, and noisy speech) introduce no new external data. They are generated deterministically at build time from the sources above, with mix policies summarized in Table 4.3.

The critical subset (Section 4.5) does not draw from any of the sources above. Every clip in it is sourced by hand from publicly accessible YouTube material and labeled manually.